A COMPARATIVE STUDY OF DATABASE TECHNOLOGIES FOR TIME SERIES DATA

Yosafe Fesaha Oqbamecail

Master's Thesis in Applied Computer Science and Engineering

Engineering Data Science Specialisation

Faculty of Technology, Environmental and Social Sciences

Western Norway University of Applied Sciences

20th August 2026

ABSTRACT

There has been an increase in the deployment of underwater sensor networks for environmental monitoring. This trend has produced a need for database systems capable of managing large volumes of time-series data efficiently. The SFI Smart Ocean project looks to create an autonomous marine observation platform which integrates underwater sensor networks with cloud-based infrastructure for use in long-term environment monitoring and industrial marine operations. As these deployments expand, choosing an appropriate database technology becomes a vital architectural decision because of the differing performance characteristics of the available database systems. This study examined the suitability of TimescaleDB, MongoDB and InfluxDB for the management of large-scale time-series data generated by representative Smart Ocean workloads. The research designed and implemented a benchmarking framework that is also reproducible. The framework created a controlled environment to evaluate selected database technologies under identical experimental conditions. Synthetic observation datasets representative of Smart Ocean sensor data was generated in progressively increasing sizes and they were loaded into each database system. Storage efficiency, ingestion performance and query performance were studied. Ingestion performance was assessed using observation throughput, execution time and data throughput. Query performance was evaluated using latest-value query latency and storage efficiency was measured via database storage growth and average storage consumption for each observation. The benchmark results showed major performance differences among the three database systems. MongoDB had the shortest ingestion execution times, the highest data throughput and observation throughput and the lowest latest-value query latency and storage footprint. InfluxDB demonstrated more balanced performance across the evaluated metrics, maintaining stable query and ingestion behaviors with moderate requirements for storage. TimescaleDB successfully supported all benchmark workloads, but generally showed greater query latency, higher execution times and larger storage requirements than the other evaluated database systems. The study concludes that the architecture of each database has a major influence on the performance characteristics of large-scale time-series data management systems. All three databases demonstrated different performance trade-offs associated with their architectures. The benchmarking framework and empirical findings are contributing evidence supporting database selection decisions for Smart Ocean and similar environmental monitoring applications.

Keywords: Time-series data, database benchmarking, Smart Ocean, underwater sensor networks, MongoDB, InfluxDB, TimescaleDB, ingestion performance, query latency, storage efficiency.

ACKNOWLEDGMENTS

I would like to thank my supervisor, Lars Michael Kristensen, for his support, feedback, and guidance throughout this thesis.

Contents

CHAPTER 1

INTRODUCTION

It is difficult to discuss global ecosystems without acknowledging the role oceans and seas play especially in climate regulation, energy production and food security [26]. Monitoring such large and dynamic environments needs advanced communication and sensing technologies [12]. Underwater Sensor Networks (UWSNs) have come up as a solution for continuous and real-time data collection in subsea infrastructure, marine ecosystems and oceanographic research [13].

The SFI Smart Ocean project is focused on creating an autonomous and flexible marine observation platform that integrates cloud-based management systems with underwater sensor networks [1]. This platform allows long-term, large-scale monitoring of underwater environments and industrial installations. The multi-parameter sensor data is collected at multiple underwater sensor sites and transmitted to surface nodes and cloud infrastructure where it is stored, processed, and analyzed.

However, underwater deployments introduce additional constraints when compared to terrestrial deployments. Acoustic communication has low bandwidth, high latency, and low reliability; and the sensors operate in extreme temperature, pressure, biofouling, corrosion and long-term electronic drift conditions. These constraints will require robust and fault-tolerant data management systems once the data arrives at land-based infrastructure.

One of the main technical challenges comes from the volume and speed of time series data generated by the sensor networks. There are typically hundreds of sensors continuously transmitting time-stamped observation across many variables per device. With many deployments, this creates thousands of datasets which require efficiency to ingest and analyze when needed, store in the long-term and access in real-time.

Traditional relational database management systems (RDBMS) offer strong consistency and mature tooling, but they are not inherently optimized for high-throughput time series workloads. In response to the growth of IoT and sensor-based systems, specialized Time Series Management Systems (TSMSs) have

emerged. These systems are designed to efficiently ingest, index, compress, and query time-stamped data streams, often offering built-in functions for time window aggregation, down-sampling, and retention management [23].

Because there are numerous database technologies, finding the right one with the right trade-offs for large scale marine sensor networks can take time. Therefore, there are no commonly accepted best models for databases regarding tradeoffs of ingestion performance, query execution time, scalability, storage efficiency, and complexity to operate. Many benchmarks for databases currently exist in the form of general use "IoT" databases or databases for financial data streams, which do not usually represent the true workloads of underwater sensor platforms.

This thesis fills this gap by providing a comparative evaluation of selected database technologies for large-scale time series data in the Smart Ocean. It evaluates various types of databases using a structured evaluation approach assessing ingestion rates, query performance, scalability act, and efficiency of storage and present the results as empirical evidence. The results of this study will provide valuable comparisons of the performance as well as identify architectural trade-offs which will help in making informed decisions on selecting a specific database for the Smart Ocean and similar sensor-driven systems.

1.1 Background of the Study

1.1.1 The SFI Smart Ocean Platform

The SFI Smart Ocean project, is a project funded by the Norwegian Research Council. The collaborative initiative involves research institutions and industry partners working together to develop intelligent and scalable marine monitoring systems [2]. The project is aimed at creating an autonomous Smart Ocean platform capable of supporting long-term underwater observation and industrial marine operations. Its research efforts span three interconnected domains.

First, it aims to deploy advanced underwater measurement and sensor technologies that can monitor the harsh subsea environment continuously. The sensors are engineered to use energy efficiently and for a long time. This means that they can collect multi-parameter data such as pressure, salinity, temperature and other environmental variables. Secondly, the project aims to solve the problem with underwater wireless sensor networks based on acoustic communication. Acoustic transmission has physical limitations, meaning that the networks have to operate under constrained bandwidth, intermittent connectivity and high latency. Due to this, data transfer becomes a major engineering challenge. Thirdly, the initiative is creating a cloud-based Smart Ocean platform that can ingest, store, process and visualize the data received from distributed sensor nodes. This platform can integrate heterogeneous data streams using standardized APIs and data formats while making sure that the data is access-

ible in the long term, available when needed and scalable. After the sensor observations make it to the surface, they are forwarded to a centralized infrastructure that is designed for reliability and efficiency.

1.1.2 *Motivation*

Once the Smart Ocean's underwater network is deployed, it will generate continuous, time series data streams. Every sensor will be able to produce timestamped measurements across different variables. As more and more sensors are introduced, the data will grow and will need long-term storage. This workload comes with data management challenges like efficient indexing of the time-stamped data, requirements for high ingestion throughput, need for long-term storage and need for real-time query support so that the data can be useful for monitoring, alerting and supporting analytical queries. The data will also need to be scalable across a growing number of sensor deployments.

Traditionally, relational database management systems (RDBMS) would provide mature transaction support and reliability but they were not optimized for time-ordered data streams. Characteristically, time series workloads require sequential writes, aggregation over windows, time-based filtering and retention management which need specialized indexing and storage strategies. Dedicated time series databases and alternative architectural approaches have come up to meet these demands. These systems use techniques such as write-optimized storage engines, time-partitioned storage, columnar compression and pre-aggregation mechanisms to better the performance of time-centric workloads [23].

However, it is yet to be established whether these database technologies are suitable for large-scale underwater sensor networks especially in marine deployments. The Smart Ocean platform needs high ingestion performance, scalability, reliability, storage efficiency and operational maintainability over long deployment periods. Picking an appropriate database technology is therefore a critical architectural decision. A systematic comparative evaluation of candidate systems is necessary to determine which technologies provide the most favorable performance and scalability trade-offs under representative Smart Ocean workloads.

1.2 Problem Statement

The Smart Ocean platform is expected to manage continuous large amounts of time-stamped data generated by a wide network of underwater sensors. In most cases, these data streams are append-heavy, they require time-based indexing and long-term storage and they have mixed workloads that involve analytical processing of historical data and real-time monitoring queries. Due to all these considerations, selecting the appropriate database technology is a major architectural challenge. Different database systems use different indexing strategies

and storage models including but not limited to time-partitioned hyper tables, relational row-based storage and document-oriented storage. These differences affect ingestion throughput, storage efficiency, operational complexity, query latency and scalability.

Even though many database technologies claim to work with time series workloads, the way they perform specially under large-scale marine sensor deployments is yet to be properly evaluated. Existing benchmarks focus on generic IoT workloads, monitoring metrics or financial trading streams which may not accurately reflect query behavior, ingestion patterns and the long-term storage demand of underwater sensor networks. This thesis looks into how different database technologies behave under Smart Ocean time series workloads. In this case, the Smart Ocean platform represents other similar data types. The objective is not to find a universally optimal database system but to evaluate and compare how the existing ones perform and the scalability trade-offs one has to make with the selected technologies within the defined operational context.

1.3 Research Questions

This research will seek to establish the database technology that provides the most favorable performance and scalability trade-offs for dealing with large-scale time series data generated from the Smart Ocean underwater sensor network. It will specifically seek to answer the following questions:

- **Research Question 1:** *How do TimescaleDB, InfluxDB, and MongoDB compare in terms of data ingestion performance when processing representative Smart Ocean time series workloads?*
- **Research Question 2:** *How do TimescaleDB, InfluxDB, and MongoDB compare in terms of latest-value query performance when processing representative Smart Ocean time series workloads?*
- **Research Question 3:** *How do TimescaleDB, InfluxDB, and MongoDB compare in terms of storage efficiency when managing large-scale Smart Ocean time series data?*

1.4 Research Methodology

To systematically evaluate database technologies for the Smart Ocean time series workload, this thesis adopts the technology evaluation methodology proposed by Brown and Wallnau [8]. This framework provides a structured approach for assessing competing technologies by identifying measurable performance deltas under representative operational conditions. The methodology consists of three sequential phases: descriptive modeling, experiment design, and experiment evaluation. Figure 1.1 illustrates the overall structure of this framework.

The descriptive modeling phase establishes the relationship between the prob-

lem domain and candidate technologies. In this thesis, the problem domain is defined by the Smart Ocean time series workload, characterized by high ingestion rates, append-heavy data patterns, time-based queries, long-term retention requirements, and scalability demands. During this phase, the workload characteristics and system requirements are formally defined. These include expected ingestion throughput, query types (e.g., time-window aggregation, range scans, real-time monitoring queries), data retention duration, storage efficiency considerations, and scalability constraints. Simultaneously, the following candidate database technologies are analyzed in terms of their architectural principles: TimescaleDB, a time-series database built as an extension of PostgreSQL, InfluxDB, a purpose-built time-series database, and MongoDB, a general-purpose NoSQL document database. The analysis includes examining their storage models, indexing strategies, compression mechanisms, clustering capabilities, and time series support features. The goal of this phase is to "situate" each technology within the Smart Ocean problem domain and identify the relevant characteristics that influence performance outcomes.

The experiment design phase translates the descriptive analysis into measurable hypotheses and benchmark configurations. Based on the workload requirements identified in the previous phase, hypotheses are formulated regarding expected system behavior. For example, column-oriented storage engines may demonstrate improved compression and analytical query performance, while specialized time series engines may exhibit superior ingestion throughput. Representative workload models are then constructed to simulate Smart Ocean data streams. These workloads emulate realistic sensor behavior, including configurable sampling rates, multi-variable measurements, and long-term data accumulation. A standardized benchmarking environment is defined to ensure fair comparison across systems, including consistent hardware configuration, controlled system tuning, and uniform dataset characteristics. Performance metrics are selected to evaluate system behavior. These include ingestion throughput, query latency, storage efficiency, scalability under increasing data volumes, and resource utilization. Clear evaluation criteria are established to measure performance deltas between candidate technologies.

In the experiment evaluation phase, the selected database systems are deployed within the defined benchmarking environment and tested under the representative workloads. Experimental results are collected and analyzed according to the predefined metrics. The evaluation includes quantitative performance benchmarking as well as compatibility assessment with Smart Ocean requirements. Observed performance deltas are interpreted in relation to architectural differences identified in the descriptive modeling phase. This structured comparison enables a systematic technology assessment rather than an isolated performance ranking. The outcome of this phase is a comparative evaluation of the candidate database technologies, highlighting their strengths, limitations, and trade-offs in the context of large-scale marine time series data management.

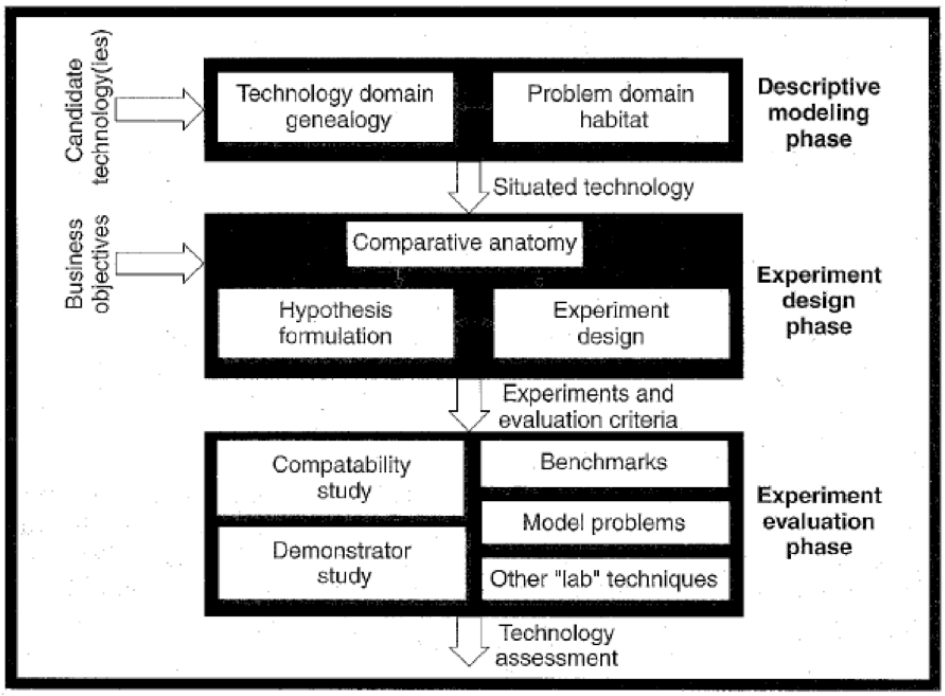

Fig. 1.1: Software technology evaluation framework (adapted from Brown and Wallnau [8]).

1.4.1 Use of AI Tools

All through the development of this thesis, artificial intelligence was employed in a limited and supportive capacity. Microsoft Copilot was used solely for spelling correction and to enhance linguistic clarity. Claude.ai was used to structure the thesis, clarify some concepts during the writing process and to debug code within the experimental framework. All research decisions, findings, and written content presented in this work are the independent contributions of the author.

1.5 Thesis Outline

The rest of this thesis is organized as follows:

1. **Chapter 2 Background and Related Work:** Provides the necessary background and context. This chapter reviews the challenges and characteristics of time series data, and surveys existing database technologies for time series data management. It discusses related work and prior research around database performance in IoT and sensor data scenarios.
2. **Chapter 3 Design and Analysis:** Outlines the design of the experimental evaluation, including the selection of database technologies, the design of test workloads, and the metrics used for assessment.
3. **Chapter 4 Implementation and Prototypes:** Describes the implementation of the experimental setup, including the deployment of database systems and the creation of test environments. Details the development of prototypes or models used for evaluation.
4. **Chapter 5 Evaluation and Results:** Presents the empirical results of the comparative evaluation of database technologies. Includes ingestion throughput, query performance, storage efficiency, and other findings.
5. **Chapter 6 Conclusion and Future Work:** Summarizes key findings and contributions, provides recommendations for the Smart Ocean platform, and suggests directions for future work.

CHAPTER 2

BACKGROUND AND RELATED WORK

The incremental and fast growth of sensor-driven systems and Internet of Things (IoT) infrastructures [4] has by necessity changed the landscape of data management. Throughout scientific, financial, industrial and environmental domains, time-stamped data streams are continuously generated at a growing scale. As discussed in Chapter 1, large-scale monitoring platforms like the Smart Ocean system create sustained append-heavy time series data which needs reliable storage, long-term retention and efficient querying. These demands have led to the creation of specialized database architectures and methods for evaluation intended to accommodate high-ingestion and time-centric workloads.

Traditional transactional datasets and time series data differ in a number of critical respects. Time series data has a temporal ordering that is intrinsic to both query behavior and interpretation. Its generation patterns are often batch oriented and append-dominated instead of being update driven [16]. The emergence of IoT systems has ramped up these characteristics, introducing multidimensional and high-velocity data streams generated from distributed sensors [10]. Consequently, conventional database architectures initially optimized for Online Transaction Processing (OLTP) workloads struggle structurally when applied to time-centric data management cases.

To address this, different architectural approaches have come up. Relational database systems now include time-based partitioning and indexing strategies to support temporal workloads. Native time series databases are designed specifically to optimize append-heavy time-range and ingestion queries. Analytical engines that are column-oriented now leverage compression and have vectorized execution improve the performance for large-scale aggregations. Document-oriented systems have included flexible schema models that can accommodate heterogeneous sensor data while cloud-managed time series platforms offer scalable infrastructural abstractions for IoT applications.

Alongside architectural innovation, methodologies for empirical evaluation have adapted to assess system performance under operational conditions that are more realistic. Comparative performance studies have measured query

latency, ingestion throughput, scalability behavior and storage efficiency across different database platforms. Workload modeling frameworks like YCSB [11] have tried to standardize experimental conditions for cloud systems. However, as will be demonstrated in the subsequent sections, architectural coverage, variations in workload realism and methodological transparency complicate comparisons across studies.

Factoring in how diverse database paradigms are and the complexity of time series workloads, a review of the existing academic research is necessary so as to situate the present research within existing scholarship. This chapter looks at existing prior work done across four interrelated domains: database architectural paradigms for temporal data management, the characteristics of time series data and IoT workloads, methodological approaches to empirical system evaluation and comparative studies of time series database performance. Using this synthesis, recurring limitations and research gaps are identified. In the analysis, it is demonstrated that even though there has been significant progress to understand time series database performance, cross-architecture comparative evaluation under realistic workload conditions is yet to be well-explored. This gap motivates the investigation undertaken in the subsequent chapters.

2.1 Time Series Data and IoT Workloads

2.1.1 *Definitions of Time Series Data*

Time series data refers to a set of data points measured and recorded over time, where the order of the data points in time is critical for understanding and analyzing the data [16]. Unlike cross-sectional data, time series data cannot be changed arbitrarily. In computer systems, time series data is typically modeled as a tuple containing a time component and one or more measurements.

Stonebraker, Çetintemel, and Zdonik [42] highlight the importance of the prevalence of continuous data arrival and window queries in stream and time-oriented databases, which differ from traditional transaction-oriented databases. Time series workloads, therefore, are characterized by window aggregation and append-dominated write patterns [16], which make them different from Online Transaction Processing (OLTP) workloads, which have characteristics like atomic transactions, complex joins, and high update frequency.

2.1.2 *Characteristics of Sensor-Generated Time Series Data*

IoT systems typically have interconnected sensors which continually collect and send out environmental or operational data [17]. These sensors produce structured high-frequency data streams which needs persistent storage and real-time accessibility. Existing literature has identified several defining characteristics of sensor-generated time series data.

Large-scale sensor networks are able to generate substantial volumes of records when they measure many parameters simultaneously [23, 41]. As the number of sensor deployments increases, ingestion rates increase. Sensor data tends to be multidimensional. One time stamp could correspond to many attributes such as vibration, salinity, temperature, pressure and many other environmental indicators. This complicates schema design and strategies for indexing especially when different sensors measure heterogeneous variable sets.

Thirdly, IoT data is often highly cardinal. Cardinality refers to the number of distinct combinations of dimensions like geographic regions, sensor identifiers, measurement categories or device types. Datasets with high cardinality increase how complex the indexing needs to be as well as the memory overhead, presenting challenges for scalability [41].

Fourthly, sensor data often needs to be stored over a long period. Industrial deployments and environmental monitoring platforms operate continuously for years. Long-term data retention is necessary to allow historical trend analysis, compliance verification, anomaly detection and predictive modeling [18]. To keep such large amounts of data imposes storage demands and pressure on query optimization mechanisms.

Finally, time series data generated by sensors tends to be append-only. Historical observations are rarely modified, and data updates primarily involve introducing new records. This append-heavy trait influences storage engine design, encouraging sequential write optimization and time-based partitioning.

2.1.3 *Identified Challenges in IoT Time Series Management*

The literature identifies many technical challenges associated with managing IoT-generated time series data. One of the major ones is storage scalability. Even with moderate sampling frequencies, IoT deployments generate substantial data volumes over a long operational period [17]. Storage systems have to support sustained ingestion rates while staying durable.

A second challenge involves query efficiency. Often, monitoring systems rely on time-based queries like retrieving measurements in specific time intervals or computing windowed aggregates [23]. When indexing and partitioning strategies are not optimized, query latency can degrade significantly with growing datasets [37]. Continuous ingestion leads to unbounded data growth. Many IoT systems implement down sampling strategies, retention policies or hierarchical storage architectures which introduce another dynamic [18]. Down sampling reduces the requirements for storage by aggregating high-resolution data into coarser time intervals.

Data consistency and reliability further complicate the management of time series data. Distributed sensor networks sometimes experience intermittent connectivity, delayed transmissions or packet loss [25]. When this happens,

time series systems have to be able to tolerate out-of-order data arrival and variable ingestion patterns without losing temporal integrity.

2.1.4 Scalability and Ingestion Constraints

In time series systems, scalability can be examined in both horizontal and vertical dimensions. Vertical scalability is about increasing computational resources inside a single node such as memory allocation, CPU capacity or disk throughput. In contrast, horizontal scalability involves distributing data across multiple nodes to accommodate higher storage volumes and ingestion rates. As discussed in an earlier section, high-ingestion environments represent a primary bottleneck in IoT deployments. Unlike read-dominated workloads, sensor-based systems tend to be write-intensive. Continuous insert operations overwhelm storage engines if indexing, logging, or synchronization mechanisms are not optimized for sequential access patterns. Excessive indexing easily degrades write performance while insufficient indexing increases query latency [16].

Distributed scaling introduces other complexities, including coordination costs, data partitioning strategies, and replication overhead. A common way to distribute load has been partitioning by device identifier or time interval, but poorly chosen partitioning schemes can cause uneven resource utilization or hot spotting. Database systems have to be able to navigate trade-offs between storage compression, query responsiveness and ingestion efficiency. In large-scale marine monitoring systems and similar deployments, these ingestion and scalability constraints directly influence decisions for database selection. Any database technology evaluation for time series workloads has to consider realistic cardinality levels, ingestion rates, distributed deployment scenarios and retention durations.

2.2 Database Architectures for Time Series Data

The need to manage time series data has led to a number of different database architectures that are aimed at enabling temporal data processing at scale. Relational database systems remain dominant, but specialized time series, columnar analytical, and NoSQL databases have been developed to overcome their limitations in scalability and efficient storage for analytical purposes. The design includes different options regarding how the data should be organized, indexed, and distributed over multiple structures. These design considerations need to be taken into account when assessing the database technologies for large sensor-based environments that require analytical as well as operational monitoring.

2.2.1 Relational Databases and Time-Based Extensions

Relational database management systems (RDBMS), notably PostgreSQL and MySQL, have traditionally excelled in handling structured data in various fields, particularly in enterprise and scientific contexts. These rely on row-oriented storage models, strong transactional guarantees, and structured query language (SQL) as the main interface for data manipulation and retrieval. Their mature ecosystems offer tooling, documentation, and operational best practices. However, relational database architectures were initially designed for online transaction processing (OLTP) environments, where workloads are composed of comparatively small transactions and involve a high volume of updates and deletes as well as point lookups [35].

On the other hand, temporal datasets accumulate continuously over time and are often looked up over bounded intervals instead of individual entries. Since the row-oriented storage engines' design goal is to optimize access to entire record sets, there may be performance restrictions when a query requires scanning a vast number of historical data or making aggregates of data for a given time interval. Correspondingly, structural optimizations were created in modern relational systems to advance their appropriateness with temporal datasets. One of such approaches is to partition tables based on time boundaries. Here, query scanning is restricted to the relevant portions of the dataset, to reduce index sizes and enhance efficient query execution. For instance, there are declarative partitions in PostgreSQL for dividing tables automatically across specified temporal boundaries, with queries being directed to access specific partitions rather than the whole dataset [37].

Further refinements are brought in via extensions such as TimescaleDB [50]. TimescaleDB is built on top of PostgreSQL and has introduced hyper tables that are abstracted over both the temporal and optional spatial dimensions. This allows for datasets to be distributed in a transparent manner across multiple partitions while maintaining the relational model, and SQL compatibility. Consequently, the scalability benefit does not depend on changing the query interface since the execution plan for temporal queries can be optimized and streamlined. However, given the potentially massive historical archives, performance limitations may remain in the relational systems. This implies that large aggregation queries need to scan entire partitions, even when only a subset of attributes is of interest. Relational databases work well because of their generality and extensive support infrastructure, but alternative designs are suggested for more efficient analytical workloads.

2.2.2 Native Time Series Database Systems

Native time series databases (TSDBs) represent a class of systems designed specifically to manage chronologically indexed datasets. Unlike general-purpose relational databases, these systems adopt storage and indexing mechanisms that

align directly with the access patterns typical of temporal data. Many TSDB implementations employ log-structured storage engines in which new observations are appended sequentially to disk structures optimized for continuous data ingestion. This sequential organization reduces disk seek operations and enables efficient compression techniques that exploit the similarity between adjacent measurements [36].

InfluxDB is a great illustration for this design philosophy through its Time Structured Merge Tree (TSM) storage engine. There, data is initially written to memory buffers and then it is periodically compacted into immutable on-disk segments that are organized by time range [19]. This strategy lowers write amplification while supporting efficiency in retrieval within specified time windows [19]. In addition, retention policies enable administrators to automatically remove outdated data, which allows long-term deployments to operate without extensive manual oversight [31].

Specialized architectures introduce certain trade-offs. Some TSDB platforms provide limited relational query capabilities, particularly for operations that require multi-table joins or complex relational constraints. What is more, the surrounding ecosystem for specialized databases may be less mature than that of long-established relational systems, which can affect the availability of tooling and operational support.

2.2.3 *Column-Oriented Analytical Databases*

Another architectural paradigm that works for large-scale analytical workloads is column-oriented database systems. Different from row-based storage models, columns organize data by attribute and keep each column independent. This design makes several performance advantages for analytical processing possible. To start with, it allows every attribute to be compressed with encoding techniques tailored to the distribution of values inside the said column. Consequently, storage requirements get significantly reduced especially when datasets have repeated or slowly changing values. Secondly, query engines are able to load just the required columns for each computation reducing input/output overhead during execution [3].

ClickHouse is an example of a modern column-oriented database intended to support large scale data. It executes using vectorized processing which evaluates operations in value batches simultaneously instead of processing records individually. This approach allows the system to rely on CPU cache locality and the SIMD capabilities of modern processors to improve query throughput for aggregation-intensive tasks [9]. Since column-oriented storage keeps each attribute physically separate, queries only need to read the columns relevant to the computation, which reduces the volume of data scanned from disk. This makes such systems well suited to workloads dominated by scans and aggregations over large historical datasets, such as those commonly found in temporal data analysis.

Even so, column stores face performance challenges in environments where data is added continuously at extremely high rates. Maintaining columnar compression structures while making frequent updates can introduce overhead which affects ingestion performance. Due to this, column-oriented databases get deployed in architectures where analytical processing and operational ingestion are separated. Then, they become downstream analytical engines for historical datasets.

2.2.4 NoSQL and Document-Oriented Approaches

NoSQL databases were introduced to address scalability and inferior schema flexibility witnessed in traditional relational systems. Semi-structured data representations are often used, which can change in form after a given period, as opposed to mandatory relational schema provisions. Document-oriented databases as MongoDB store information in the form of JSON-like documents with the ability to set nested fields in each record or remove some attributes for one record [30]. This is crucial in situations where devices or sources measure different things, and the type of measurement data can change as new measuring devices are added.

For chronologically indexed datasets, MongoDB has recently formalized specialized collections. The collections create observations stored in internal buckets that contain related measurements. This is achieved by reducing the metadata requirements and limiting index modifications, thus enhancing storage optimization and write efficiency for chronologically indexed datasets [29]. In addition, document databases implement distributed scaling with sharding, where datasets are divided into partitions within groups of nodes in a cluster. Sharding makes it possible to scale to bigger data sizes and workloads by spreading the data and the computation across multiple machines.

Despite these advantages, NoSQL systems generally adopt less strict consistency models compared to relational databases. [44] notes that while RDBMSs enforce strong ACID consistency, NoSQL systems typically relax these guarantees to achieve higher performance and scalability for workloads with large numbers of concurrent users.

2.2.5 Monitoring and Metrics-Focused Systems

Monitoring platforms like Prometheus [39] and Graphite [15] represent a specialized category of systems designed primarily for infrastructure observability and operational metrics collection. These systems prioritize simplicity, reliability, and real-time alerting capabilities.

Prometheus uses a pull-based model of data collection where monitored services expose endpoint metrics which are periodically scraped by the monitoring server [39]. The data collected is then stored in a local time series storage engine that is optimized for rapid retrieval of recent data [38]. This architecture makes

it possible to evaluate alerting rules and dashboard queries fast in operational monitoring environments.

Graphite, on the other hand, uses a different design centered around hierarchical metric naming conventions [15]. The metrics get stored in fixed-size files in formats that maintain pre-aggregated values at many resolutions [15]. This structure simplifies long-term trend visualization but it also introduces constraints on query flexibility since the data has already been aggregated during storage.

Systems made for monitoring environments often emphasize predictable performance for alert evaluation instead of general-purpose analytical exploration. Consequently, their applicability to complex sensor analytics especially those using multi-dimensional datasets or needed extended historical analysis, has to be assessed carefully. These systems offer valuable capabilities for operational visibility but it remains to be seen whether they are suitable for large-scale scientific data analysis.

2.3 Core Properties of Time Series Database Systems

The preceding discussion of time series workloads and database architectures indicates that systems designed for temporal data management tend to share a number of recurring architectural properties. Although implementations vary across relational extensions, native time series engines, column-oriented databases, document-oriented platforms, and managed cloud services, the literature suggests that effective time series database systems exhibit several core characteristics shaped by the structural demands of append-heavy, time-ordered data.

First, append-optimized storage is fundamental. Time series workloads require a lot of sequential insert operations rather than transactional updates, as noted by [23]. Because of this, efficient time series systems prioritize high-throughput write paths, frequently employing log-structured storage mechanisms or time-partitioned tables to reduce write amplification and limit random disk access.

Second, time-based partitioning is necessary for both performance and scalability. Dividing data into temporal segments reduces index size, improves the efficiency of time-range queries, and simplifies retention management. Relational extensions like hyper tables, as well as native time series engines, rely on time-aware segmentation to optimize ingestion and retrieval operations.

Third, efficient execution of range queries and windowed aggregations is essential. Time series analysis commonly involves retrieving measurements within bounded intervals and computing statistical summaries [23]. Database architectures must therefore support rapid scanning of contiguous temporal segments, whether through row-based indexing strategies or column-oriented compression [23], with some systems additionally leveraging vectorized execution to accelerate analytical queries [9].

Fourth, because support for data lifecycle management is critical, mechanisms such as down sampling and partition expiration are necessary. Effective time series systems must provide automated management of historical data while maintaining uninterrupted ingestion workflows [18]. Besides, time series datasets frequently include numerous device identifiers, geographic attributes, or measurement types. Storage and indexing strategies must accommodate these dimensions without introducing prohibitive memory overhead [41].

Finally, both vertical and horizontal scalability remains a defining property. As discussed in previous sections, ingestion rates and dataset size increase proportionally with deployment scale. Systems must therefore sustain growing write loads and query demands through efficient resource utilization and, where applicable, distributed data partitioning.

These core properties do not prescribe a single architectural solution; rather, they establish criteria by which time series database systems can be evaluated. Differences in how individual systems implement append optimization, partitioning, compression, and scalability mechanisms provide the basis for meaningful comparative analysis. These core properties form the foundation for the experiment.

2.4 Data Modeling and Indexing Strategies for Time Series Databases

2.4.1 Time Series Data Models

Efficient time series data management not only depends on the database architecture but also on the way the data is modeled within the storage system. A time series data model defines the logical structure used to represent, retrieve and store temporal observations. It is different from traditional relational datasets in that time series data emphasizes chronology and continuous accumulation.

A common way to represent time series data is through the timestamp-value model, where each record contains a timestamp and one or more related measurements. In sensor-based environments, additional contextual attributes are often needed to explain the source of the observation. Many systems, therefore, use an extended timestamp-dimension-value model, in which dimensions represent metadata like sensor identifiers, device types, geographic locations, or measurement categories [21]. This format lets systems organize observations by both time and contextual attributes.

In practice, time series data models are created using either narrow schemas or wide schemas. Narrow schemas store each measurement as a separate record with a timestamp, a measurement identifier, and a value. This structure allows for flexible schema changes and makes indexing easier across different data streams. Wide schemas, on the other hand, store multiple measurements linked

to a single timestamp within the same record. This method reduces storage use and can improve query performance when retrieving multiple attributes at once.

In practice, time series data models are created using either narrow schemas or wide schemas. Narrow schemas store each measurement as a separate record with a timestamp, a measurement identifier, and a value. This structure allows for flexible schema changes and makes indexing easier across different data streams. Wide schemas, on the other hand, store multiple measurements linked to a single timestamp within the same record [47]. This method reduces storage use and can improve query performance when retrieving multiple attributes at once.

The decision between narrow and wide schemas involves balancing storage efficiency, schema flexibility, and query performance. Narrow schemas are often preferred in systems that handle various sensor measurements, where different devices may produce different variables over time. However, wide schemas can simplify indexing when measurements have a consistent structure.

2.4.2 *Indexing Strategies for Timeseries Data*

For fast time series data retrieval and analysis, efficient indexing is necessary. Since these datasets grow to billions of records over a long period, indexing strategies have to support efficient query execution and high ingestion throughput. Different from traditional workloads where queries often target individual records, time series workloads include range queries, window aggregations and filtering by metadata (Jensen et al. [23]).

Fundamentally, time series systems use time-based indexing. Because many observations are ordered by time, indexing structures are designed to mirror them and allow efficient retrieval within specified temporal intervals. Many systems keep indexes on timestamp columns to speed up range scans and enable querying that retrieves data within defined windows of time. In relational extensions like TimescaleDB, this functionality gets used through composite indexes which combine timestamps and metadata attributes like device identifiers.

It is common to filter by device or measurement in large-scale sensor environments. Because of this, time series databases often embrace multi-dimensional indexing strategies which combine categorical and temporal attributes. They allow queries to efficiently retrieve data associated with specific sensors or geographies while still keeping it chronological. Some specialized databases even employ alternative mechanisms for indexing meant for high-cardinality datasets to reduce overhead while supporting efficiency.

Large-scale analytical systems also use skip indexes or sparse indexing structures to allow query engines to rule out irrelevant data blocks during scanning operations [43]. These mechanisms improve query performance across large

datasets by significantly reducing the amount of data read during query execution.

2.4.3 Storage Layout and Partitioning Strategies

Many time series database engines organize data into continuous storage segments instead of storing it as independent records. This way, incoming data is grouped into blocks representing a bounded temporal range or a specific stream. Each block will have a sequence of time stamped data streams stored sequentially in disk structures or memory. The grouping helps to reduce random disk access and allow queries to process large portions of data through sequential scans.

Segment-based storage structures typically have metadata summaries describing what is contained in each block. These summaries include attributes like the maximum and minimum timestamp in each block, the number of observations stored as well as statistical summaries of different values. As one executes a query, the database engine examines the metadata before accessing the actual data. If a block's timestamp data is outside the query interval, then the entire block is skipped. This technique is called block punning and it reduces disk reads when scanning historical datasets

Segmented storage makes it possible to run compaction processes in the background. As new observations get appended to the system, the data is often organized into small segments which later merge into larger blocks through compaction. Compaction reorganizes stored data to improve its locality and remove redundancy. Since these operations happen asynchronously, they let systems optimize storage layout without interrupting query activity or ongoing ingestion.

Another advantage to segment-based layouts is how compatible they are with immutable storage structures. After a segment is written to completion, it is usually treated as read-only. These segments make concurrency control simpler because multiple queries can get access to the same data without synchronization conflicts. They also allow database engines to run maintenance operations like reorganization on older segments while new data still arrives.

The relationship between query execution, segmented storage structures and ingestion buffers is shown in Figure 2.1. The diagram shows how incoming measurements get accumulated into sequential storage segments that are scannable during analytical queries. Segment-oriented storage layouts provide a structural mechanism for managing large volumes of chronologically ordered data.

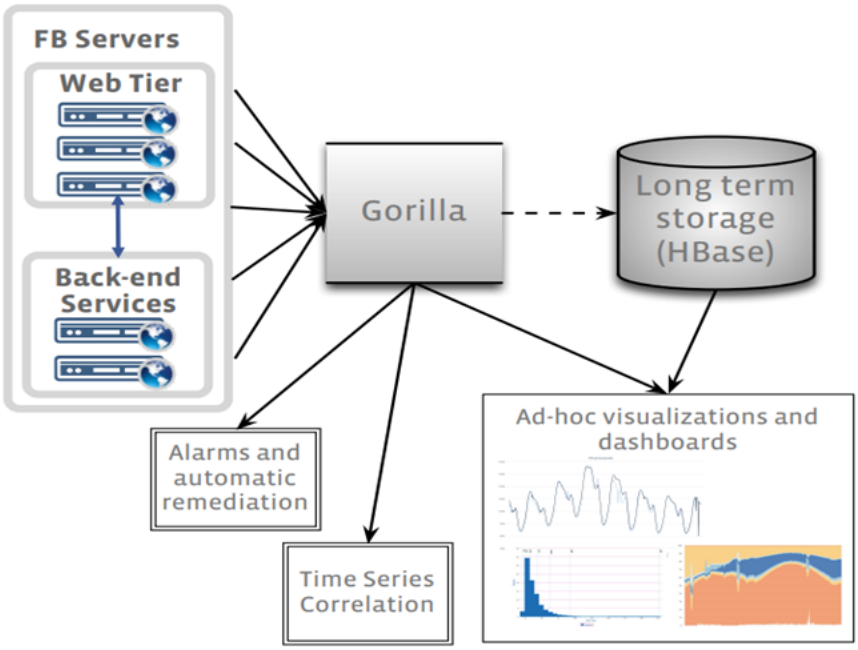

Fig. 2.1: Architecture of the Gorilla time-series monitoring database illustrating ingestion buffers and segmented storage structures used for efficient processing of temporal data. Adapted from (Pelkonen et al. [36]).

2.4.4 *Query Optimization in Time Series Databases*

2.4.5 *Query Optimization in Time Series Databases*

Time series analysis often involves operations that combine observations across temporal intervals instead of retrieving the records individually. Queries will typically compute moving averages, rolling statistics or summary metrics across sliding time windows. Since these operations often deal with large amounts of data, database systems have to use specialized query optimization strategies to keep performance acceptable. One way they do this is through window functions which allow queries to calculate over ordered data subsets defined by boundaries of time. Rather than scanning repeatedly, execution plans can instead process observations incrementally as new records enter the query window, obtaining each window's answer from the previous window's answer rather than recomputing it from scratch [14]. This approach is well suited to analytical operations like cumulative sums, moving averages and anomaly detection across different time intervals.

Another optimization strategy often used is the implementation of continuous queries [6]. These queries automatically compute certain aggregations when new data arrives, updating the results incrementally instead of recomputing them from the whole dataset. Monitoring systems will often use this approach to keep up-to-date summaries of system metrics without running expensive analytical queries over and over. Continuous query mechanisms help reduce

computational overhead for metrics that are frequently accessed.

Majority of time series databases support materialized rollups where historical data gets pre-aggregated into low-resolution summaries [46]. Rather than storing just the raw observations, systems will periodically generate aggregated views that represent larger time intervals – like daily summaries. When queries request long-term historical trends, the database is able to retrieve results from summarized tables instead of scanning high-resolution raw data. This reduces the query execution time while preserving the ability to analyze long-term data.

In large-scale monitoring environments, some systems will use approximate query processing techniques to speed up analytical queries over very large datasets [5]. Approximations estimate aggregate values using sampled data subsets instead of evaluating the whole dataset. These estimates may sacrifice small degrees of precision, but they often provide better performance for exploratory analysis and tasks that require visualization.

2.5 Performance Evaluation of Database Systems

Evaluating database systems for time series workloads calls for systematic experimental design, controlled workload modeling, and carefully selected performance metrics. Unlike purely theoretical architectural analysis, empirical performance evaluation is concerned with quantifying how systems behave under representative operational conditions. In the context of large-scale sensor data management, such evaluation must consider ingestion throughput, query latency, scalability behavior, and storage efficiency while maintaining fairness across heterogeneous architectures. This section lays the theoretical framework for the benchmarking conducted in the next chapter.

2.5.1 *Principles of Database Performance Evaluation*

Taipalus [44] emphasizes that meaningful evaluation demands a clear definition of not just workload characteristics, but also performance objectives and system configuration parameters. Without well-defined experimental conditions, performance comparisons run the risk of reflecting configuration bias instead of architectural differences. For time series systems specifically, evaluation has to account for time-range queries, append-heavy writes and long-term retention [24].

Another basic principle involves transparency and repeatability. Experiments need to be designed in such a way that results are reproducible in similar conditions. This includes documenting things like storage configurations, hardware specifications, system tuning parameters and indexing strategies. Any differences in default configurations could influence how performance is observed especially across systems with distinct architectural paradigms. In heterogeneous environments like comparisons between relational extensions and native

time series systems, careful control of the variables of the experiment is necessary. Each system may prioritize different strategies for optimization including distributed storage models, write buffering mechanisms or compression algorithms.

2.5.2 *Workload Modeling for Time Series Systems*

Workload modeling is very critical in evaluating database performance. A workload represents the operations pattern executed against a database including query types, insert rates and data distributions. For time series systems, workloads have to reflect realistic injection patterns and query behavior typical of sensor and IoT deployments.

Cooper et al. [11] introduced the Yahoo! Cloud Serving Benchmark (YCSB) as a framework for evaluating NoSQL systems under configurable read-write workloads. Originally designed for key-value stores, the concept of workload-driven evaluation has since influenced broader research in database benchmarking. In the context of time series data, workload models generally emphasize sustained insertion rates alongside periodic aggregation queries based on range.

Workload realism matters in sensor-based systems. Industrial and environmental monitoring platforms tend to produce data at regular intervals but network variability can introduce delayed arrivals or burstiness. A realistic workload model should consider these things. It must also consider data retention duration. Short-term evaluations may not show the performance degradation that happens as datasets grow over months or years. Scalable evaluation requires progressive data accumulation to echo long-lived deployments.

2.5.3 *Evaluation Metrics in Time Series Studies*

Performance evaluation of time series databases typically relies on several quantitative metrics:

- Ingestion throughput - measures the records inserted per second under sustained load. This metric is necessary for append-heavy sensor systems. High ingestion capacity ensures that incoming data streams can be stored without backlog or data loss.
- Query latency - measures the time required to run representative queries, like retrieving measurements within a time range or computing aggregate functions. Latency may vary depending on whether queries target recent or historical data.
- Storage efficiency - evaluates the efficiency of a database in compressing and storing time series data. Compression ratios and storage overhead influence long-term operational costs, especially in large-scale deployments.
- Scalability - refers to the way the system behaves as workload intensity or data volume increases. It includes both vertical scaling and horizontal

scaling.

- Resource utilization metrics – These metrics, including CPU usage, memory consumption, and disk I/O behavior, provide additional insight into system efficiency. High resource utilization under moderate workloads may indicate architectural inefficiencies or tuning limitations.

Khelifati et al. [24] argue that evaluation should consider trade-offs rather than focusing on a single metric. A system with superior ingestion throughput may exhibit higher query latency or increased storage overhead. Therefore, comparative evaluation must interpret performance across multiple dimensions.

2.5.4 Limitations in Existing Comparative Studies

Although many studies have evaluated database systems under time series workloads, several limitations are evident in the literature. First, many evaluations focus on a limited subset of systems, often comparing only native time series databases while excluding relational or document-oriented alternatives. This restricts architectural diversity in analysis.

Second, some studies rely on synthetic workloads that do not accurately reflect real-world sensor environments. Simplified workloads may not capture high cardinality, bursty ingestion patterns, or long-term retention challenges. Third, configuration transparency is sometimes limited. Inconsistent tuning across systems may introduce bias, particularly when default settings vary significantly.

Fourth, cloud-native managed services such as Amazon Timestream are underrepresented in academic comparisons, despite their growing adoption in industrial IoT deployments. Finally, few comparative studies explicitly address marine or environmental monitoring contexts, where long-term data accumulation and reliability requirements differ from financial or infrastructure monitoring systems.

These limitations indicate the need for a structured and transparent comparative evaluation of heterogeneous database architectures under representative sensor workloads. The present study addresses this gap by systematically evaluating TimescaleDB, InfluxDB, MongoDB and Amazon Timestream, under controlled experimental conditions designed to reflect large-scale time series data management requirements.

CHAPTER 3

DESIGN AND ANALYSIS

This chapter discusses the experimental framework used in this research and how it will evaluate database technologies for large-scale time series data management. The previous chapter considered the theoretical foundations of time series databases, their architectures and characteristics. This chapter builds on that to translate those insights into structured and reproducible experimental design. The goal is to define the way the comparative evaluation will be done in a way that is methodically sound, transparent and aligned with the requirements of the specific context of the research.

The evaluation will focus on assessing the scalability and performance of the selected database systems under representative time series workloads. Instead of basing the research on abstract comparisons or isolated performance claims, this study adopts an experimental approach that is controlled and in which a number of database technologies are subjected to identical workloads under similar conditions. This ensures that any differences observed in the performance can be attributed to the architectural and implementation characteristics instead of inconsistencies in the setup of the experiment.

To meet this goal, the chapter defines key components of the experimental design. It establishes its evaluation framework and methodological approach. This includes defining the scope of comparison, mentioning the relevant performance dimensions and outlining the principles that are used to guarantee fairness and reproducibility. Then, it describes how different database systems were selected to be included in the study alongside their rationale. Thirdly, it details the dataset design and workload models used to simulate realistic data generated by sensor-based systems.

This chapter also specifies the benchmarking approach and the experimental environment include the configurations of the system, deployment conditions and hardware setup. These elements are necessary for making sure that performance measurements remain consistent and comparable across systems. It defines the metrics that are used to assess system performance, including query latency, ingestion throughput, storage efficiency, scalability behavior and re-

source utilization. Each metric is operationalized based on how it is measured and interpreted inside the experimental framework.

Finally, the chapter outlines the experimental procedure and discusses the potential ways the design is limited. This includes considerations related to system configuration constraints, dataset realism and generalizability of the results. These factors are addressed to make sure the research is balanced and transparent.

3.1 Experimental Design Framework

To evaluate database systems for time series data management, one needs a structured methodological approach that ensures comparability, consistency and reproducibility. To achieve this, this study adopts the evaluation framework proposed by Brown and Wallnau [8], which offers a systematic process for assessing competing technologies on the basis of measurable performance metrics. This framework is especially fitting for comparative system evaluation because of its emphasis on the identification of relevant performance dimensions, designing representative experiments and analyzing the observed differences between technologies. The framework is organized into three primary phases: descriptive modeling, experiment design and experiment evaluation. Every phase plays a critical and separate role in structuring the research process and making sure the evaluation stays grounded both in its theoretical understanding and empirical evidence. In the context of this study, the phases are mapped as follows:

The descriptive modeling phase sets up the relationship between the problem domain and the technologies being evaluated. It involves identifying the major characteristics of the workload and the system requirements which influence performance outcomes. As addressed in Chapter 2, the descriptive modeling showed the properties of time series data, the architectural features of database systems and the problems associated with sensor workloads. The insights from that chapter inform the selection of performance metrics and the design of the experimental workloads used in this research.

The experiment design phase is the focus of this current chapter. It translates the conceptual understanding that was developed in the descriptive modeling into a concrete plan for experimentation. This phase will define the scope for evaluation, selected database systems and design the representative workloads and datasets. It will also specify the metrics to be used to assess performance. A key objective here is to make sure all systems get evaluated under consistent conditions to enable meaningful comparison. To achieve comparability, each database is subjected to a similar dataset, similar workload patterns and a similar hardware environment. That way, differences in performance can be attributed to system behavior instead of external factors. To do this, system configuration is considered carefully, including the indexing strategies, data

ingestion methods and query execution settings. Where possible, default configurations mirror typical deployment scenarios while making sure no system is unfairly disadvantaged because of misconfiguration.

During the experimental design phase, the formulation of evaluation criteria is considered. Instead of using just one performance metric, this study adopts a multi-dimensional evaluation approach which considers different aspects of system performance. These include query latency, which measures how responsive the system is to analytical queries, ingestion throughput, which is about the system's ability to handle continuous data input and storage efficiency, which is about how effective data compression and storage utilization are. The research may also consider scalability, which is about the way performance changes with increases in data volume and resource utilization which offers insight into system efficiency under load.

Throughout the experiment design, principles of transparency and repeatability are considered. All aspects of the setup including system configurations, hardware specifications and workload definitions are explicitly documented. This makes sure that the results can be reproduced under similar conditions, which is a fundamental requirement for empirical research. Besides, experimented are conducted a number of times to account for differences in system performance and the results are averaged to offer a more reliable representation of system behavior.

The final phase of the Brown and Wallnau [8] framework, experiment evaluation, involves executing the defined experiments, collecting the data and analyzing the results. This phase will be addressed in subsequent chapters where selected database systems are evaluated based on metrics defined in this chapter. The analysis is intended to identify performance trade-offs between systems and understand how differences in architecture influence observed behavior under time series workloads. The framework adopted strengthens the validity of the findings and offers a clear basis for interpreting the results in relation to the research questions defined earlier.

3.2 Selection of Database Systems

Database	Highlights	License	Best For
InfluxDB	SQL and InfluxQL; no temporal joins; no matviews, last-write; object store native (Core); Explorer app	MIT/Apache, commercial tiers	Recent-data metrics & IoT
TimescaleDB	PostgreSQL SQL (no TS); no temporal joins; matviews with dedup on write; tiering via Tiger Cloud only; cloud console	Apache 2.0 plus TSL	Time-series in PostgreSQL
MongoDB	Document-oriented (BSON); native time series collections since v5.0; automatic bucketing and columnar compression; writable non-materialized views backed by an internal collection	Server Side Public License (SSPL)	Flexible-schema applications with time-series data (IoT, logs, event data)
QuestDB	SQL and time semantics; full temporal joins; matviews with dedup on write; tiered Parquet (Enterprise); console and dashboards	Apache 2.0 plus Enterprise	Capital markets, energy, aerospace & robotics
TDengine	SQL; ASOF plus window joins; matviews (TSMA) with dedup; tiering via Enterprise; taosExplorer	AGPL-3.0 plus Enterprise	Industrial IoT & edge
ClickHouse	SQL dialect; ASOF join only; matviews with eventual dedup; S3 tiering (OSS); /play console	Apache 2.0 plus Cloud	Large-scale analytical scans
kdb+	q and SQL (KDB-X) and Python; ASOF plus window joins; no matviews, DIY dedup; proprietary storage; KX Dashboards, pgwire	Proprietary, free tiers	HFT & capital markets
DolphinDB	SQL plus scripting language; ASOF plus window (functions); no matviews, ingest dedup; proprietary storage; GUI and web dashboards	Proprietary, free Community	Capital markets, industrial IoT
Prometheus	Kubernetes-native monitoring; PromQL query language; single-node (Thanos/-Mimir for horizontal scaling)	Apache 2.0	Kubernetes-native monitoring
VictoriaMetrics	Prometheus long-term storage; MetricsQL (PromQL-compatible); horizontal scaling with free clustering	Apache 2.0	Prometheus long-term storage

Database	Highlights	License	Best For
Apache Druid	SQL (via Calcite); horizontal scaling; real-time stream analytics	Apache 2.0	Real-time stream analytics
Graphite	Custom query language; limited scalability	Apache 2.0	Legacy monitoring

Table 3.1: Comparison of time series databases: highlights, licensing, and best-fit use cases. Compiled from QuestDB [40], Tiger Data (Timescale) [45], and MongoDB [27].

For this study, the selection of database technologies is guided by the need to evaluate systems which represent distinct architectural approaches to time series data management while still staying practically deployable within the experimental environment scope. Instead of attempting to evaluate all possible time series databases, this study stays focused by selecting a few representative systems that allow for meaningful comparisons across different philosophies of design. Three systems are picked for the initial phase of the evaluation – MongoDB, TimescaleDB and InfluxDB. These systems were picked because together, they represent different paradigms for managing time series data – native time series database architectures, document-oriented storage and relational time-series extensions. This diversity ensures the study can examine how fundamentally different design decisions can influence system behavior under identical workload conditions.

TimescaleDB [48] is picked to represent relational database systems extended for time series workloads. It is built on top of PostgresSQL so it retains its relational model and SQL interface while adding time-based partitioning and optimizations for performance that are tailored to temporal data. It is included to allow the study to find out how far a relational system can adapt to handle time-stamped data without abandoning established principles.

InfluxDB [20] is added to the list as a purpose-built time series database intended to specifically serve high throughput ingestion and improve efficiency of time-based querying. It has a storage engine, built-in retention mechanism and query model which are optimized for temporal workloads. Investigating InfluxDB provides insight into the limitations and performance advantages of specialized systems which prioritize time series operations over general-purpose use.

MongoDB [28] represents document-oriented NoSQL approaches to time series data management. It has a flexible schema design and very recently introduced time series collections to allow it to accommodate heterogenous sensor data with diverse structures. It has been included in the study to provide insight into how a schema-flexible, general-purpose system performs when applied to structured time series workloads.

The selection of the above systems was also influenced by the practical considerations related to integration and implementation. All three databases offer mature Python libraries including pymongo for MongoDB, psycopg2 for TimescaleDB and influxdb-client for InfluxDB. These libraries make it possible to keep the interaction patterns consistent across systems within the experimental framework. Because there are stable client libraries, performance measurements can be conducted programmatically and reproducibly.

Notably, this study is designed to be extensible. While the initial evaluation will focus on these three systems, the framework for experimentation allows for the inclusion of more databases at a later stage. This modularity is made possible by standardized dataset formats, standardized measurement procedures and

standardized definitions for the workloads. Consequently, future systems can be added into the evaluation without requiring major changes to the overall design. The goal of this selection is not to find one superior database system, but to examine how different architectural approaches work under similar workload conditions. A focus on a small but representative set of systems makes sure the evaluation remains both analytically meaningful and manageable.

3.3 Dataset Design and Characteristics

How the dataset is designed in this study is critical to the experimental framework because it directly influences the relevance and validity of the study. Instead of relying on pre-existing benchmark datasets, this study creates a synthetic dataset that mirrors the characteristics of time series data from sensor deployments. That way, the study has precise control over data properties like structure, distribution and volume, making sure that the dataset aligns with the operational context defined earlier.

The dataset is made to simulate data generated by a sensor network similar to the Smart Ocean platform. Every record in the set represents a time-stamped record produced by a sensor devise. Each record is structured to include a sensor identifier, timestamp field and many measurement attributes that correspond to operational or environmental variables. These attributes could include values like salinity, temperature, pressure or other sensor-specific metrics. This is done to reflect the multidimensional nature or data from real-world sensors.

To support comparative evaluation across the different database systems, the dataset is defined in a way that can be mapped consistently to each model. For relational systems like TimescaleDB, the dataset is saved as a structured table with columns that are clearly defined. For InfluxDB, the same data is mapped to tags, measurements and fields with the different sensor identifiers represented as tags to support efficiency in filtering. In MongoDB, the records get stored like documents with field structures that are flexible to allow measurement attributes to get nested inside each observation.

Datasets are programmatically generated to make sure they are reproducible and scalable. Synthetic data will give the study control over key parameters such as the number of sensors, how frequently data is generated and the duration of the simulation observation period. The idea is to be able to adjust these parameters to produce datasets of different sizes and enable the evaluation of system behavior under varying data volumes. For instance, increasing the number of sensors or sampling frequency creates higher ingestion rates while stretching the time horizon raises the total dataset size.

To make sure that experiments are consistent, the dataset is generated once and then reused across different systems. This removes variability that is introduced by differences in data content and makes sure each system is evaluated under identical conditions. The process of data generation is documented fully

including the scripts used and parameters applied to enable reproducibility.

3.4 Workload Design

This workload design matters to the validity of the evaluation because it defines the operations through which database systems get compared and exercised. While the dataset informs the structure and content of the stored data, the workload specifies the way the data is written, accessed and analyzed. Within the context of time series systems, workload design has to reflect both continuous ingestion of new data and query execution which retrieves and processes historical data. This study's workload is made to represent two primary classes of operations – query workloads and ingestion workloads. These categories capture the dominant usage patterns in time series applications.

3.4.1 Ingestion Workload

The ingestion workload simulates how sensor-generated data arrives. Data is entered into each database system in sequential batches to reflect the nature of time series data. Instead of entering records individually, the study batches insertion strategies so as to approximate realistic data pipelines where data gets transmitted in grouped intervals instead of as isolated entries. Batch size gets treated like a configurable parameter, letting the evaluation examine how different systems respond to different ingestion patterns.

To make sure that operations are fair across systems, ingestion operations get implemented with the respective database drivers in Python. MongoDB interactions get handled through pymongo, TimescaleDB uses psycopg2 and InfluxDB is accessed via its Python client library. The standardized interaction layer makes sure that all systems get subjected to comparable application-level behavior. Ingestion performance gets measured in terms of throughput, set as the number of records that get successfully inserted per unit time.

3.4.2 Query Workload

Besides ingestion, workload includes queries that are designed to evaluate how the system performs under normal monitoring and analytical scenarios. The queries are made to reflect common operations done on time series data.The query workload is divided into categories based on how they are intended to function. Time-range retrieval is where the system retrieves all observations inside of a specified interval for a given set of sensors or sensor. These queries test how able the system is to scan and filter time-stamped data.

The second category deals with aggregation queries where statistical summaries like counts, minimums, averages and maximum values get computed over a set time window. These queries are necessary for reporting and monitoring applications and need efficient processing of large data volumes. A third category

includes multi-dimensional filtering queries where data gets retrieved based on a combination of metadata attributes and time constraints. These queries typically add a layer of complexity because they require the system to evaluate many filtering conditions at the same time.

Query workload is executed after there is sufficient volumes of ingested data to make sure that the measurements for performance reflect realistic dataset sizes. The queries are repeated a number of times to account for differences in execution time and the results get averaged to produce stable performance estimates.

3.4.3 Workload Consistency and Control

For workload design to be effective, it must also factor in consistency across all evaluated systems. For this, the same sequence of operations will be used in each database with equivalent query definitions and identical datasets. Differences in the query syntax between systems get resolved via careful mapping to make sure that each query runs the same logical operation even if the implementation underneath it is different.

Workload execution is controlled so that there is no interference between query operations and ingestion unless that is explicitly required. Query workloads and ingestion are executed separately to isolate their performance characteristics. This makes sure that the measurements do not get affected by concurrent interactions between workloads. The study is performed this way to make sure the basis for the evaluation is consistent and realistic.

3.5 Experimental Environment and System Configuration

The environment of the experiment refers to the set of conditions under which all the database systems get deployed and evaluated. When the environment is well defined, it makes performance comparisons reproducible, fair and attributable to characteristics of the system instead of external factors. This section describes the software configuration, hardware setup and deployment approach that is used in the study.

3.5.1 Hardware Environment

All experiments are run inside a controlled hardware environment to get rid of variability from differences in computational resources. The evaluation is done on a single-node system set with sufficient memory, power and storage capacity to support the defined workloads. This is a deliberate design choice intended to isolate the intrinsic performance characteristics of each database system. Distributed deployments may be common in large-scale production environments but introducing multiple nodes would add a layer of complexity related to synchronization, network communication and cluster management.

The focus on a single node configuration makes it possible for the study to ensure that performance differences reflect the efficiency of each system's storage engine, query processing mechanism and indexing strategy. Hardware specifications including memory capacity, CPU type, storage type and number of cores are documented to make the evaluation reproducible. Where possible, storage devices that have consistent performance characteristics get used to minimize variability in disk I/O behavior.

3.5.2 Software Environment

Each system is deployed with stable and widely adopted versions to make sure the evaluation is reliable and consistent. The software environment includes the Operating System, database engines and the client libraries used for interaction. InfluxDB, MongoDB and TimescaleDB are installed and configured accordingly. The default configurations become the baseline with minimal adjustments made only according to need to make sure there is the time series functionality or ensure compatibility. That way, the experiment reflects realistic deployment scenarios where systems often get used with standard configurations instead of tuned setups. The experimental framework is run in Python which works like the orchestration layer for query execution, ingestion. Python is chosen because it has an extensive ecosystem of database drivers and it is suitable for automation and scripting. A single programming language across all systems makes sure that there is operational consistency.

3.5.3 Database Configuration

Even though the default configurations are utilized as the baseline, certain database-specific settings get adjusted to align with the study requirements. For instance, TimescaleDB is configured to enable time-partitioned tables and make sure that data is organized in a way that is consistent with how it will be used. InfluxDB gets configured with necessary measurement definitions and MogoDB uses time series collections to optimize query performance and storage. Indexing strategies get defined consistently across systems to support query workload. In the framework, time-based indexes are also created to enable efficient by timestamp filtering while extra indexes may be applied to sensor identifiers or other attributes that are frequently queried. Care is taken to make sure that no system is unfairly advantaged or disadvantaged and that similar logical structures get used across databases.

3.5.4 Execution Control and Isolation

To make sure there is consistency, experiments are run under conditions which minimize external process interference. Background applications are limited during testing and the resources of the system monitored to make sure there is no unexpected load that can affect performance measurements. Each experiment is run independently and the database system is reinitialized or reset

between runs according to need. This is to prevent caching effects or residual data from influencing the next experiment. Where the experiment requires caching behavior, it is explicitly accounted for in the experimental design.

3.5.5 Considerations for Reproducibility

Reproducibility is necessary in any empirical evaluation. To support it, all aspects of the experimental environment are recorded including software versions, hardware specifications, execution procedures and configuration settings. The scripts that are used for ingestion, data generation and query execution are preserved and may be reused to replicate the experiments.

3.6 Evaluation Metrics

This study evaluates database systems using a set of carefully defined performance metrics that capture different aspects of system behavior for time series workloads. Instead of relying on just one measure of performance, the study uses several benchmarks which reflect different operational requirements of large-scale sensor data systems. Every metric is defined the way it operates, with clear procedures for measurement to ensure comparability and consistency across all the studied database technologies.

3.6.1 Ingestion Throughput

This metric considers the rate at which the system can accept and persist incoming data. It is a measure of the number of records that are successfully inserted into the database per unit time, often expressed in records per second. It is a necessary metric for time series systems where continuous data streams have to be ingested with minimal delay or data loss. To measure it, data is added into each database in controlled batches and the total time used to complete the insertion gets recorded. Throughput gets calculated as the ratio of the number of inserted records to the total time used. The measurements are taken after an initial warm up to make sure that the results reflect a steady state system rather than initialization overhead.

3.6.2 Query Latency

Query latency is a measure of the time it takes for a database system to process a query and return results. It accounts for the system's response time for analytical and monitoring activities, which is a critical factor for applications that depend on the availability of data in a timely manner. Latency is determined by a timing method that measures the beginning and end of a query execution process. For a driver-based execution, the beginning time is measured immediately before the query execution, and the end time is measured after the results are received by the application. The difference between these times is the query latency, which includes the client and network times.

3.6.3 Storage Efficiency

Storage efficiency measures how well a database system uses storage space in handling its time series data. It is defined as the ratio of physical storage space used by the database system for its data to the input data set's size. In calculating storage efficiency, the input data set's storage space is obtained after the ingestion process is complete, including all data structures maintained by the database system, which include indexes and metadata. The input data set's size, on the other hand, is obtained before the ingestion process, allowing for a direct comparison of the input data set and the stored data space. This is a critical factor in a database system, especially in long-term usage, as data is constantly being added over time. A database system that has a higher storage efficiency is ideal for usage in a storage space constraint or a long-term retention scenario.

3.6.4 Scalability

Scalability is a measure used to determine the changes in system performance as the amount of data or intensity of work is increased. Scalability is not defined by a specific numerical value, but by a trend in system performance over different amounts of data. In this particular study, the concept of scalability is analyzed by increasing the size of the data set used in the experiment, repeating the ingestion and query evaluation processes, and analyzing the results obtained. This way, the study can determine the trend in the data ingestion throughput of the system. A system is defined to be scalable if its performance is stable or decreases gracefully in response to increased workload intensity. A sudden decrease in system performance may imply a problem in storage structures, indexing, or resource management.

3.6.5 Resource Utilization

Resource utilization offers an insight into the efficiency of a database system with regards to its ability to effectively utilize resources. The metrics include CPU usage, memory, and disk I/O. Resource utilization is tracked for both ingestion and query execution. High resource usage can imply poor design or configuration of a system, while low resource usage with high performance can imply effective optimization. Resource utilization is not a key aspect of the study but offers useful information for understanding performance results. For instance, two systems can have similar performance but drastically different resource requirements.

3.6.6 Composite Interpretation of Metrics

The evaluation does not depend on any one metric alone to determine system performance. Rather, the results are interpreted across the identified metrics to notice any trade-offs between different system behavior aspects. A database that

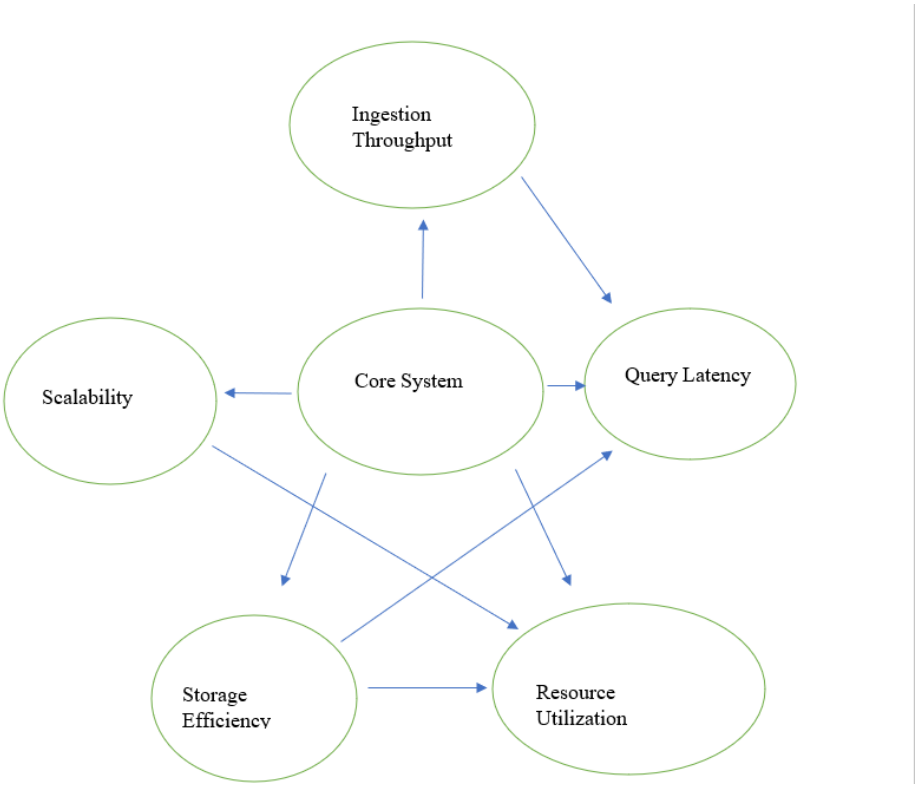

Fig. 3.1: Conceptual relationship between performance metrics in time series database evaluation

does well in terms of ingestion throughput could have a higher query latency or a lower storage efficiency. These trade-offs have to be considered based on the intended application.

The relationship between the evaluated performance metrics is not independent, as improvements in one dimension may introduce trade-offs in another, consistent with multi-metric evaluation approaches in time series database benchmarking [24]. This interaction is illustrated in Figure 3.1.

3.7 Data Analysis Methods

The analysis of the results of the experiment needs a structured approach that guarantees reliability, consistency and meaningful interpretation of the observations. This section details the methods used to process, aggregated and compare the data that is collected in the evaluation. The goal is to turn the raw observations into interpretable results while lowering the influence of measurement noise and variability.

3.7.1 Repetition and Averaging

To make the measurements more reliable, each experiment is performed multiple times in the same environment. This reduces the impact of the system's transient states and the presence of background processes, among other sources

of indeterminism. The results from each run are combined using the average function. This gives a more reliable view of the system's performance than using a single run's result. In addition, the range of the results is used to determine the consistency of the system's performance.

3.7.2 Normalization of Results

To make the results more comparable, some of the results, such as ingestion throughput, are normalized to a standard scale. This is done, for instance, by normalizing the ingestion throughput relative to the best system's throughput. This is important because it makes it easy to compare the performance of systems whose performance is measured in different ranges.

3.7.3 Comparative Analysis

The main analysis technique is the comparison of performance metrics between various database systems. For instance, for a given performance metric, results are grouped into comparative tables and graphical displays, making it easy to spot performance differences between various database systems. Comparative analysis is done along various axes, including data size, workload types, and complexity. By conducting a multidimensional analysis, it is possible to spot trends in database system behavior, for example, how performance changes with increasing data sizes.

3.7.4 Trend Analysis

Trend analysis is another analysis technique used to analyze performance metrics against changing workload conditions. For example, ingestion throughput can be plotted against data size to understand performance changes with time. Similarly, query latency can be analyzed against different types of queries or time intervals. Understanding trends in performance metrics, makes it possible to understand the performance and reliability of various database systems. For instance, a constant trend in performance metrics is a guarantee of a reliable database system.

3.7.5 Handling Variability and Outliers

It is possible that performance measurements occasionally have outliers resulting from system interruptions, measurement anomalies or resource contention. These outliers get identified through inspection of statistical summaries and repeated measurements. Where possible, the outliers are excluded from the analysis to avoid a distortion of the results. However, the exclusion is carefully justified to make the process transparent. In cases where that variability is significant, it gets reported alongside average values to give a complete picture of system behavior.

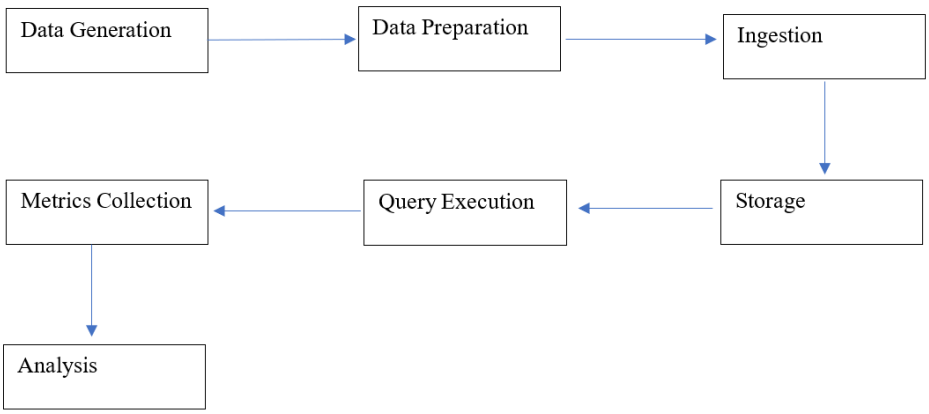

Fig. 3.2: Experimental workflow illustrating the sequence from data preparation to performance analysis

3.7.6 *Experimental Procedure*

The experimental procedure outlines the steps taken to carry out the evaluation process. The study's formalization of this process guarantees that all database systems are subject to the same environment, hence a consistent performance measurement process. The process begins with the initialization of the experimental environment. The database systems are set up, initialized, and verified to ensure correct functioning before the experiment. The database systems are reset to a clean state before running the experiment, where necessary, to avoid any effects from previous runs of the experiment.

The sequence of operations involved in the experimental procedure is illustrated in Figure 3.2. The workflow captures the progression from dataset preparation through ingestion, query execution, and performance measurement. It is adapted from the Time Series Benchmark Suite (TSBS) repository (Timescale, n.d.-b).

After initialization, the data set is fed into the database system through a predefined workload. The insertion of data into the database occurs in batches, with the process's timing being recorded. The process begins with a small warm-up phase to allow the database system to settle before the performance measurement process begins. The initialization process of the database systems is avoided during the experiment to ensure a fair evaluation of the database systems performance.

3.8 Threats to Validity and Design Limitations

The experimental design outlined in this chapter aims to make the experimentation fair and the experimentation environment controlled. However, there are some limitations and potential threats to validity which must be acknowledged. Identifying the limitations is essential to the interpretation of the results accurately and to coming to useful conclusions.

One potential threat to internal validity comes from any necessary differences in

system configuration. Even though efforts are made to make all the configurations standard across database systems, each system is unique and has different tuning parameters and optimization strategies. Going with the default or adjusting configurations minimally may not fully reflect the optimal performance for each system. At the same time, extensive tuning could bring bias into the equation if the tuning is not done consistently. The study mitigates this risk by using a uniform configuration strategy and documenting all used settings.

Another limitation pertains to the use of synthetic data. Synthetic datasets are preferrable because they allow precise control over the characteristics of the data but they may fail to capture all the complexities of real-world sensor data like missing values, irregular sampling patterns and unpredictable data bursts. Because of this performance observed under the experiment conditions may differ from behavior in real world applications. However, the design of the dataset is set to approximate the key characteristics of time series workloads so that the basis of evaluation is reasonable.

While there is a rationale for the use of a single-node experimental environment, it introduces extra limitations. The setup allows for controlled comparison of the way the systems behave, but it does not capture the complexities of distributed deployments in which factors like replication, network latency and load balancing play a major role. This may mean that the results won't fully generalize to large-scale distributed systems.

Another consideration for the research is measurement accuracy. Timing measurements obtained via application-level instrumentation could include overhead from network communication or client-side processing. Even though this reflects realistic scenarios for use, it may not isolate the internal performance of the database engine.

The selection of database systems may also limit external validity. This study focuses on a subset of the available technologies and the results may not map onto systems that are not included in the evaluation. However, deliberate effort has been made to select databases that represent diverse architectural approaches to allow for meaningful insights into different design paradigms.

Finally, workload design choices may affect how performance is observed. The workload has been constructed to reflect typical time series operations but variations in ingestion strategies or query patterns could produce different results. This study addresses this by defining the workloads clearly and then applying the consistently across different runs and systems.

3.9 Synthesis of Design Decisions

The experimental framework design presented in this chapter reflects an intentional effort to balance methodological rigor with practicality. Each component of the design from workload constriction to database selection and performance

measurement, is informed by the need to enable fair and meaningful comparison of database systems under time series workloads.

One central design decision is running the experiments in controlled environments where all systems get evaluated under identical conditions. This is done to make sure that performance differences are directly attributable to system behavior instead of external variability. Using a single-node configuration helps to anchor this objective by isolating the characteristics intrinsic to each database system even though it adds limitations to distributed scalability analysis.

How the database systems are selected reflects an emphasis on architectural diversity. The study includes relational, document-oriented approaches and native time series to examine the way different design philosophies influence performance. This diversity improves the analytical value of the study. At the same time, the workload and dataset designs are structured to reflect realistic usage scenarios while guaranteeing control over experimental variables. The use of synthetic data provides flexibility and reproducibility and defining query and ingestion workloads ensures that each system is evaluated under representative conditions. Separating query and ingestion phases makes clear analysis possible for each performance dimension without any interferences.

Performance metrics are defined in ways that capture many aspects of system behavior including scalability, throughput, latency, storage efficiency and resource utilization. This approach acknowledges that no one metric is enough to fully characterize system performance and allows for the comparison to be more nuanced.

The experimental procedure and the methods for data analysis are meant to ensure consistency, transparency and repeatability. Repeated measurements, systematic analysis techniques and structured data collection contribute to reliable results and support the validity of the conclusions this study draws. Taken together, these decisions for design create a coherent and robust framework to evaluate database systems for time series data management. The framework also provides a basis for the experimental results provided in subsequent chapters.

CHAPTER 4

IMPLEMENTATION AND PROTOTYPES

This chapter documents the implementation of the benchmarking framework that was developed to evaluate the ingestion performance of InfluxDB, MongDB and TimescaleDB when storing time-series environmental observation data. The design of the implementation was intended to offer a controlled and reproducible environment where the three database systems could be subjected to identical workloads and then evaluated with a common set of performance metrics. The framework for benchmarking has a dataset management component, a database abstraction layer that's implemented via repository classes, a component for benchmark execution responsible for workload timing and generation and a benchmark result repository used to store performance measurements. These components work together to ensure automated dataset loading, execution of ingestion workloads, collection of the performance measurements and the storage of benchmark results for subsequent analysis.

To guarantee consistency across the different experiments, each database system was deployed inside a containerized environment using Docker. Separate repository implementations were created for MongoDB, TimescaleDB and InfluxDB to allow each database to be accessible through its native client while keeping a uniform benchmarking workflow. A dedicated benchmark database was also set up to persist benchmark results and facilitate later comparison and evaluation. The datasets used throughout the benchmark were created from synthetic environmental observation records representing time-series sensor measurements. Many dataset sizes were created to evaluate the way ingestion performance changes with an increase in data volume. The datasets were changes into appropriate formats for each database system while keeping the same underlying information content. This approach made sure that each database was evaluated with equivalent workloads.

The implementation also has mechanisms for dataset loading, connection handling, benchmark configuration management, database initialization, throughput calculation, execution timing and result persistence. Special consideration was given to make the process repeatable by isolating benchmark runs, clearing data that was previously inserted and recording benchmark metadata alongside

performance measurements. The rest of this chapter describes the architecture of the benchmarking framework, the preparation of the benchmark datasets, database environment configuration, the implementation of the benchmark execution workflow and the performance metrics that were collected during experimentation. The implementation details create the foundation for the evaluation and analysis presented in the next chapter.

4.1 System Architecture

4.1.1 Overall Benchmarking Framework

To make sure that the comparison of MongoDB, TimescaleDB and InfluxDB is fair and repeatable, a unified benchmarking framework was created. The framework was intended to isolate the ingestion performance of the individual databases while making sure that all systems processed the same datasets under identical testing environments. Instead of creating different benchmarking implementations for each database, a common framework was used to standardize benchmark execution, data loading, result storage and performance measurement. The benchmarking framework consists of an orchestration layer, a database abstraction layer, the benchmark result storage layer and a data generation and loading pipeline. These components create a structured environment for running ingestion benchmarks and collecting performance metrics consistently across the evaluated database systems.

Benchmark orchestration layer : This layer is the central control component of the framework. It does the job of coordinating the execution of benchmarking experiments. Dedicated benchmarking scripts were used for TimescaleDB, MongoDB and InfluxDB but all the scripts follow the same workflow for execution. For each benchmark run, the orchestration layer carries out the following tasks:

- Loads the dataset metadata from the dataset size catalogue
- Loads the corresponding dataset from disk
- Sets up a connection to the target database
- Performs warm up operations specific to each database when required
- Runs ingestion operations with predefined batch sizes
- Measures execution time with high-resolution timers
- Calculates throughput metrics
- Stores the results from each benchmark in the BenchmarkDB repository
- Clears the target database before the next benchmark iteration

This design makes sure that all the database systems get evaluated with identical

benchmark logic to reduce the possibility of implementation-specific bias influencing the results.

Database abstraction layer: This layer was implemented to provide a consistent interface for interacting with the different database technologies. Each system was represented by a dedicated repository class that included connection management, data insertion, querying and cleanup operation. The abstraction layer had a MongoDBRepo, InfluxDBRepo and TimescaleDBrepo. Each repository interacts with a different underlying database technology, but they expose similar functionality including:

- Database connectivity verification via ping operations
- Single-record insertion methods
- Batch insertion methods
- Data cleanup and deletion operations
- Query execution functionality

This approach separates benchmark logic from details specific to database implementation. Because of this, modifications to database interaction mechanisms can be made independently without affecting the workflow for benchmark execution. For example, MongoDB batch insertion is run using the native insert_manu() operation provided by its driver while TimescaleDB uses PostgresSQL's execute_values() mechanism and InfluxDB uses its batch write API. Despite these differences in implementation, all repositories offer a common interface for benchmark execution.

Benchmark result storage: A separate PostgresSQL database, BenchmarkDB, was created to store benchmark outputs independent of the databases being evaluated. Storing results externally helps prevent the benchmarking process from influencing the performance characteristics of the systems getting tested. BenchmarkDB is run using SQLAlchemy and has a dedicated table named ingestion_benchmark_results. Each execution creates a record with metadata about the experiment and the measured performance outcomes. The stored attributes were:

- The name of the database system & version
- Dataset identifier & size
- Record count
- Insert batch size
- Total execution time
- Throughput in observations per second, bytes per second, kilobytes per second and megabytes per second
- Number of performed operations

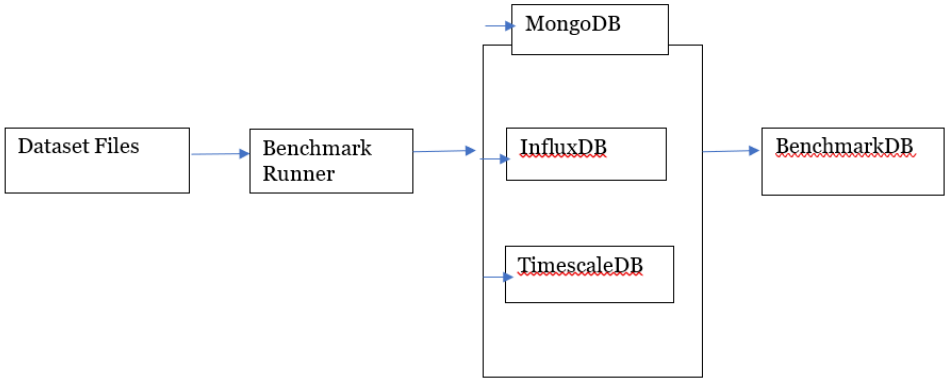

Fig. 4.1: Overall architecture of the benchmarking framework

A centralized benchmark repository helps make consistent storage possible, enables retrieval and supports the analysis of benchmark results across many experimental runs.

Data generation and loading pipeline: The benchmarking framework is based on a structured data generation and loading pipeline. Before benchmarking, datasets were generated and transformed into suitable formats for each target database system. The generated datasets were stored in different directories each corresponding to its database:

- `Data/mongodb_data/`
- `Data/influxdb_data/`
- `Data/timescaledb_data/`

MongoDB datasets were stored as JSON documents while TimescaleDB and InfluxDB datasets were stored as CSV files representing the different observations. In the benchmark execution, the framework loads the necessary dataset from disk, parses the data into the necessary in-memory representation and then passes the records to the corresponding repository for insertion. Dataset metadata like record count and storage size get loaded separately from the dataset_sizes.csv file and are later used for throughput calculations. Separating metadata management, dataset storage and benchmark execution improves reproducibility and allows for the use of identical datasets across all database systems.

Figure 4.1 illustrates the overall architecture of the benchmarking framework:

Inside of this architecture, the benchmark datasets get loaded by the benchmark runner which coordinates execution across the three database systems. Each database processes identical datasets under equivalent experimental conditions. The performance metrics collected in execution are subsequently stored in BenchmarkDB to be later analyzed and compared.

4.1.2 Project Structure

The benchmarking framework was made into a modular project structure to improve reproducibility, maintainability and separation of concerns. Instead of placing all benchmarking logic in one codebase, functionality was divided into separate directories responsible for data storage, configuration management, result analysis, database interaction and thesis documentation. Only the directly relevant components to benchmark execution are discussed here.

Configuration directory (configs/): This directory contains configuration files used to establish connections to the evaluated database systems and the benchmark result repository. Separate configs/ files are maintained for InfluxDB, MongoDB, TimescaleDB and BenchmarkDB. This approach helps the database connection parameters be modifiable without needing changes to the benchmark implementation. The configuration layer also betters portability by enabling the benchmarking framework to get deployed across different environments while keeping the benchmark logic consistent.

Data directory (data/): The data/ directory stores all used datasets during benchmarking. Separate subdirectories are kept for MongoDB, InfluxDB and TimescaleDB to accommodate the required data formats for each database system. Besides the benchmark datasets themselves, the directory had metadata files used to describe dataset characteristics like record counts and storage sizes. This metadata gets loaded during benchmark execution and is used to calculate throughput metrics and validate dataset consistency across experiments. The data directory works as the primary source of benchmark input data.

Source code directory (src/): This directory contains the implementation of the benchmarking framework. It includes repository classes, database clients, benchmark execution logic, logging utilities, configuration utilities and benchmark result management components. The source code is arranged according to database technology letting each database system maintain its own repository implementation while adhering to the common benchmarking workflow earlier described. The src/ directory is the core execution layer of the framework.

Notebook directory (notebooks/): The notebooks/ directory contains Jupyter notebooks used for exploratory data analysis, to validate benchmark outputs and to generate visualizations that are used in later chapters of this thesis. The separation of analytical notebooks from the benchmark execution code allows for experimental analysis to be performed without changing the underlying benchmarking framework. This contributes to reproducibility and supports iteration. all processed analytical outputs, figures and visualizations were generated and stored directly within the notebooks themselves.

Thesis report directory (thesis-report/): This directory contains the supporting materials for the thesis and the manuscript for report preparation. Keeping the report inside the same project repository makes sure the benchmark implementations, figures, experimental results and documentation stay synchronized

throughout the research process. The structure supports traceability between the implemented benchmarking framework and results presented in the final dissertation as well.

4.2 Benchmark Dataset Preparation

4.2.1 Observation Data Model

The benchmarking experiments were run using a common observation data model representing marine sensor measurements. A standardized observation structure made sure that identical information is stored and processed by TimescaleDB, InfluxDB and MongoDB, enabling a fair comparison of ingestion performance across the database systems. Each observation represents one environmental measurement recorded by a sensor at a specific time and location. In addition to the measured value itself, the observation also has metadata describing the source of the measurement, the data quality indicators and the physical location where it was recorded. This structure reflects the characteristics of real-world ocean monitoring systems where sensor readings come accompanied by contextual information necessary for analysis and interpretation.

The observation model has eleven fields which capture temporal information, sensor and node identifiers, geographical coordinates, quality assessment information and measurement details. Table 4.1 presents the fields included in one observation record:

The timestamp field is the primary temporal attribute used to record the exact time when the measurement was collected. Because all evaluated databases support time-series data, this field is necessary for data organization and indexing. The longitude and latitude fields represent the geographical location of the observation. The coordinates allow measurements to get associated with specific monitoring sites and supports future spatial analysis. The node source identifier and node source fields describe the monitoring platform collecting the data. A monitoring node often hosts many sensors, making node-level metadata necessary for traceability and system management.

Sensor source identifier and sensor source fields identify the specific sensors responsible for generating the measurement. They are included to allow observations from different sensor types to get stored inside the same dataset while maintaining source information. Unit, value and parameter fields together describe the measured environmental phenomenon. For example, a temperature sensor could produce a sea water temperature value expressed in degrees Celsius. Quality codes have data quality indicators used to assess reliability and validity of the observations. Such quality metadata is typically included in scientific and environmental monitoring systems to support downstream data analysis and validation.

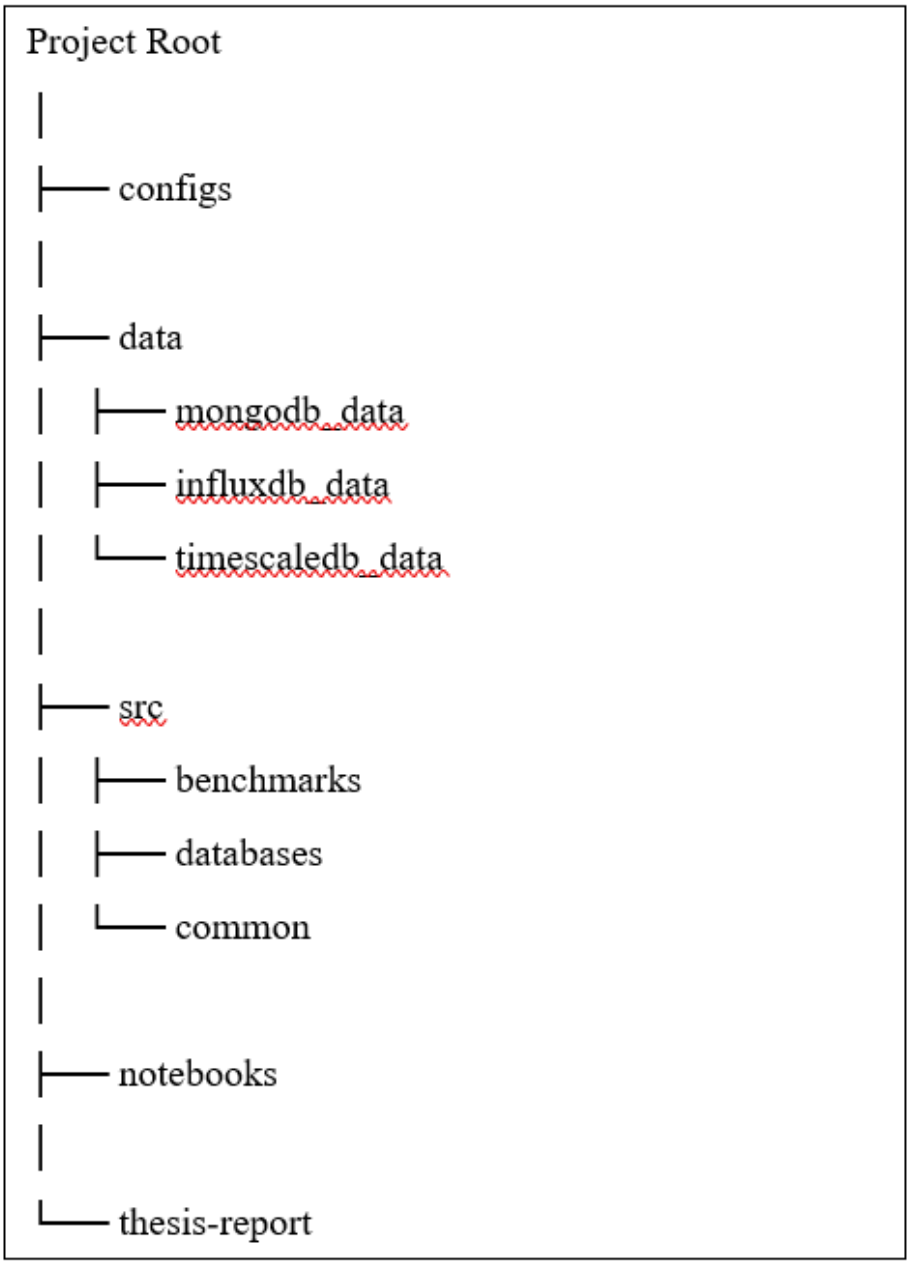

Fig. 4.2: High-level project structure of the benchmarking framework

Field	Description	Example
`sensor_source`	Name of the sensor producing the measurement	Aanderaa temperature probe
`sensor_source_id`	Unique identifier of the sensor	sfi_smart_ocean;demo;d1;1 AANDERAA_Temperature
`parameter`	Environmental parameter being measured	sea_water_temperature
`value`	Numerical value of the observation	16.71
`unit`	Measurement unit associated with the observation value	degrees_C
`time`	Timestamp showing when the observation was recorded	2025-06-01T12:00:00Z
`node_source`	Name of the observation platform or monitoring node	Smart Ocean Node
`node_source_id`	Unique identifier of the monitoring node	sfi_smart_ocean;demo;d1;1
`latitude`	Latitude coordinate of the observation location	60.090717
`longitude`	Longitude coordinate of the observation location	5.263733
`quality_codes`	Quality control indicators describing the reliability of the observation	[0]

Table 4.1: Observation data model

This observation model makes it possible to maintain a consistent benchmark dataset structure across influxDB, MongoDB and TimescaleDB. That way, all differences observed during benchmarking can be attributed to database behavior instead of variations in data representation.

4.2.2 Dataset Generation

A series of synthetic environmental observation datasets was generated to evaluate database ingestion performance under varying workloads. The datasets were designed to conform to the observation model discussed in section 4.3.1 while offering controlled and repeatable benchmark inputs for the databases. Synthetic data was picked to make sure all benchmark experiments were done under identical conditions. Using generated observations helped to eliminate inconsistencies that come from missing values, irregular sampling intervals or different data quality commonly found in real world environmental monitoring datasets. The data also allowed the benchmark datasets size to be precisely controlled, making systematic evaluation of database performance across different data volumes possible.

The raw synthetic observations were generated using JSON Generator [32], a web-based tool that produces randomized data from a user-defined JavaScript template. This allowed the creation of realistic observation records with controllable field structures and value ranges, matching the schema required.

Generated observations represent environmental sensor measurements and include node, temporal, spatial, quality-control and sensor attributes. Once generated, the datasets were transformed into storage formats required by each target database system while preserving the underlying information content. Instead of benchmarking database performance using one dataset, a collection of progressively larger datasets was made. This approach makes it possible for ingestion performance to be evaluated under increasing workloads and enables the observation of how the database system scales as the volume of incoming data grows. Sixteen benchmark datasets were generated, ranging from very small datasets with five observations to larger ones with fifty thousand observations. The associated storage requirements and dataset sizes were recorded in the dataset_sizes.csv metadata file and used in benchmark execution to calculate throughput metrics. Table 4.2 presents the benchmark datasets used throughout the study:

Batch ID	Dataset Name	Observations	Size (Bytes)	Size (KB)	Size (MB)
1	batch_1_5_obs	5	1,071	1.05	0.001
2	batch_2_10_obs	10	2,141	2.09	0.002
3	batch_3_25_obs	25	5,349	5.22	0.005
4	batch_4_50_obs	50	10,692	10.44	0.010
5	batch_5_100_obs	100	21,385	20.88	0.020
6	batch_6_250_obs	250	53,459	52.21	0.051
7	batch_7_500_obs	500	106,924	104.42	0.102
8	batch_8_1000_obs	1,000	213,835	208.82	0.204
9	batch_9_2000_obs	2,000	427,720	417.70	0.408
10	batch_10_3000_obs	3,000	641,522	626.49	0.612
11	batch_11_5000_obs	5,000	1,069,189	1,044.13	1.020
12	batch_12_7500_obs	7,500	1,603,854	1,566.26	1.530
13	batch_13_10000_obs	10,000	2,138,394	2,088.28	2.039
14	batch_14_20000_obs	20,000	4,276,864	4,176.62	4.079
15	batch_15_30000_obs	30,000	6,415,152	6,264.80	6.118
16	batch_16_50000_obs	50,000	10,691,719	10,441.13	10.196

Table 4.2: Benchmark database sizes

Progressively larger datasets were used for a number of reasons. First, they help the evaluation of ingestion performance across a wide range of workload sizes. Small datasets offer insight into system behavior during low-volume ingestion scenarios while larger datasets simulate operational environments that are more demanding. Secondly, increasing dataset sizes enable the investigation of scalability characteristics. A database could perform efficiently while processing hundreds of observations but show reduced throughput with the increase in the volume of incoming data. Using many dataset sizes provides a more complete understanding of performance behavior than a single benchmark dataset. Dataset progression also makes it possible to identify potential bottlenecks related to storage management, write operations, buffering mechanisms, transaction handling and indexing overhead. These effects only become visible as the dataset size grows. Finally, standardized datasets ensure that

all the evaluated databases are evaluated through identical workloads. This consistency is necessary to achieve fairness.

4.2.3 Database-specific dataset formats

Even though the benchmark used the same environmental observation data across the different database systems, the dataset was represented differently to match database storage models. The data structures were made according to the recommended data modeling approach for each database while keeping the same underlying information content.

MongoDB dataset format: MongoDB stores data as JSON-like BSON documents. Here, each document represented a measurement event from an ocean monitoring node at a specific point in time. A document had:

- Node information
- Geographic location
- Timestamp
- Different sensor observations

Individual sensor observations had a value, unit, parameter name and quality code. The dataset structure made it possible to embed many related observations inside one document, reducing the number of write operations necessary during ingestion. Table

Each sensor observation contained a parameter name, value, unit and quality code. The dataset structure allowed multiple related observations to be embedded within a single document, reducing the number of write operations required during ingestion. Table 4.3 presents the structure of a MongoDB document. (In the benchmark dataset, each MongoDB document contained five sensor observations collected from the same monitoring node and timestamp.)

Field	Description
`source`	Monitoring node name
`source_id`	Unique node identifier
`location`	Geographic coordinates
`time`	Observation timestamp
`observations`	Array of sensor observations
`meta`	Additional node metadata

Table 4.3: Structure of a MongoDB Document

TimescaleDB dataset format: TimescaleDB stores information using a relational table structure. Because of this, each environmental observation was represented as a separate row. The dataset was stored in a table containing spatial, temporal and sensor measurement attributes. This structure produced one database row for every individual sensor measurement appearing as shown in Table 4.4 below:

time	parameter	value	unit
2025-01-01 00:00:00	sea_water_temperature	18.68	degrees_C
2025-01-01 00:00:00	sea_water_salinity	35.45	PSU
2025-01-01 00:00:00	dissolved_oxygen	155.80	umol L-1

Table 4.4: An example representation for TimescaleDB

```
ocean_observations,
node_source=Node1,
parameter=sea_water_temperature
value=18.68,
latitude=60.090717,
longitude=5.263733
2025-01-01T00:00:00Z
```

Fig. 4.3: Conceptual representation of an InfluxDB record

InfluxDB dataset format: Typically, influxDB stores time-series data as measurements with fields, tags and timestamps. Before insertion, observations from the dataset were converted into observation objects and then changed into InfluxDB point objects. Within the implementation, these attributes, node_source, node_source_id, sensor_source, parameter and unit were stored as tags. Value, longitude, latitude and quality_codes were stored as fields. Each point was written to the measurement called ocean_observations using precise timestamps to the nanosecond. Figure 4.3 is a conceptual representation of the record:

The use of fields and tags enables InfluxDB to optimize retrieval and indexing of time-series measurements while maintaining efficient storage of sensor values. While all three database systems stored identical environmental information, their internal representation was different. MongoDB grouped many sensor observations into one document, TimescaleDB stored observations like individual relational records and influxDB converted observations into time-series points with timestamps, tags and fields. The differences reflect the underlying design philosophy for each database system and are expected to influence ingestion performance during benchmarking.

4.3 Database Environment Configuration

4.3.1 Containerized deployment

To make benchmarking repeatable and fair, all database systems were deployed with docker containers orchestrated via Docker Compose. Containerization was done to offer a standardized execution environment in which all the databases could be configured, started and managed consistently across benchmark runs. Docker Compose was used to defined and coordinate the necessary services via declarative configuration files. Instead of installing each database directly on the host OS, every database instance was executed inside its own isolated container. This approach made sure that the benchmark framework interacted with individual databases through a controlled and reproducible environment. The decision to use a container was motivated by three considerations – reproducibility, isolation and environmental consistency.

A key objective of benchmarking is the ability to reproduce experimental results. Docker Compose makes it possible to define infrastructure configurations as code which means that the same database versions, service configurations and network settings can be recreated when the benchmark is executed. This approach lowers variability introduced by manual procedures for installation and allows future researchers to recreate the benchmarking environment with little configuration effort. Maintaining database configurations inside version-controlled deployment files keeps the experimental setup repeatable and transparent.

The different database systems were deployed inside independent containers. The separation prevented software dependencies, runtime configurations and libraries from interfering with one another. Isolation was especially necessary because TimescaleDB, MongoDB and InfluxDB, all employ different communication protocols, storage engines and runtime requirements. Containerization made sure that each database operated inside its intended environment while staying accessible to the benchmarking framework via defined network interfaces. Isolated containers also made maintenance tasks like clearing datasets, restarting services and resetting experimental conditions between benchmark runs possible.

Variations in operating system configurations, dependency management and software versions can all influence benchmarking results. Docker Compose helped to minimize the sources of variation by making sure that all benchmark executions used the same container images and runtime configurations. As a result, differences observed during benchmarking were more likely to reflect the characteristics of the database systems instead of inconsistencies in the surrounding execution environment. This contributes to the internal validity of the experiments and adds to the reliability of the performance measurements obtained. Figure 4.4 illustrated the containerized deployment architecture that was used during benchmarking.

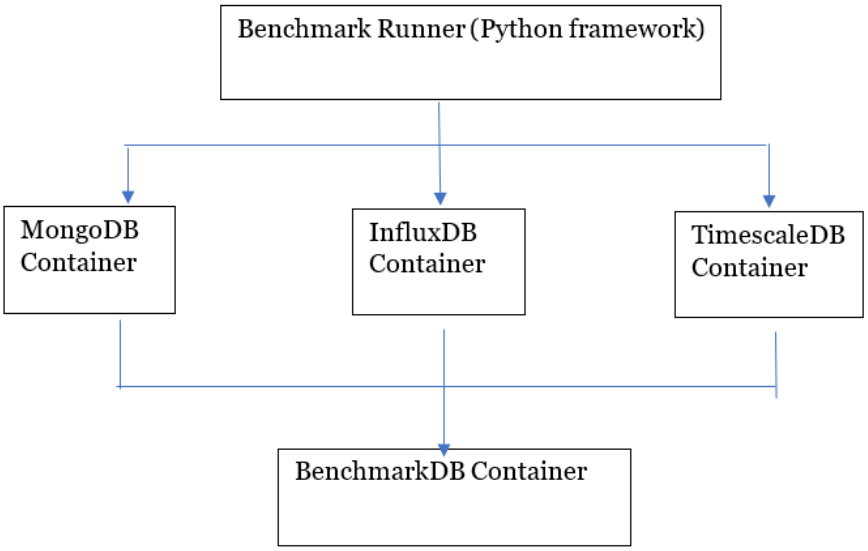

Fig. 4.4: Containerized Database Deployment Architecture

The use of containers provided a stable and reproducible foundation for executing ingestion benchmarks across the database systems while keeping the experimental conditions consistent throughout the study.

4.3.2 *MongoDB Configuration*

MongoDB was deployed in a container within the Docker Compose environment. The configuration was intended to provide an isolated and reproducible deployment while making sure that database initialization and access control were automatically established at system startup. The benchmark used the official MongoDB docker image specified in the Docker Compose configuration as:

```
mongodb:
    image: mongo:8
```

The official image was used to ensure consistency across executions and provide a standardized environment without needing manual installation on the host machine. The container exposed MongoDB's default service port (27017), which was mapped to a configurable host port via environment variables. This way, the benchmarking framework could communicate with the database using consistent network interface no matter the host environment.

Authentication was enabled via environment variables supplied at container initialization. The MongoDB root username and password were created automatically at the first launch of the container.

Environment variables helped to separate sensitive credentials from source code and configuration files while letting different deployment environments use independent authentication settings. This approach aligns with standard container security practices and made sure that benchmark operations were run

against an authenticated database instance.

The database was configured to run initialization scripts automatically at startup via the Docker entrypoint initialization mechanism.

```
volumes:
  - mongo\_data:/data/db
  - ./init-mongo:/docker-entrypoint-initdb.d:ro
```

The init-mongo directory was mounted as a read-only volume and provided initialization scripts executed when the container was created. The idea here was to enable the automatic preparation of the benchmark database environment without needing manual intervention after deployment. Automating initialization meant that the benchmark could recreate the database environment consistently across different experimental runs.

A dedicated Docker volume was attached to the MongoDB container:

```
volumes:
  - mongo\_data:/data/db
```

The volume persisted MongoDB files independently of the lifecycle of the container. It made sure that benchmark data stayed available even if the container was recreated or restarted. Persistent storage also made repeated benchmark executions and result verification possible.

To make sure the database was available before execution, a container health check was set up:

```
healthcheck:
  test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
```

Periodically, the health check sent a ping command to the MongoDB server to verify that the database was ready to accept connections. This helped lower the risk of benchmark execution starting before the database service was fully initialized.

4.3.3 TimescaleDB Configuration

Timescale was the relational time-series database platform used in this study. Its system was also configured using a Docker container but based on PostgreSQL 17 with its extension pre-installed. The reason for the configuration was to benefit from the database's traditional relational functionality but also be able to make optimizations to manage large volumes of data. The TimescaleDB service was deployed with the official image:

```
timescaledb:
  image: timescale/timescaledb:latest-pg17
```

The image combines the PostgreSQL relational database engine with TimescaleDB extension to make it possible to integrate time-series capabilities dir-

ectly into a standard SQL database environment. Database creation parameters and authentication credentials were supplied via environment variables during container initialization.

PostgreSQL as the underlying platform allowed the benchmark framework to interact with the TimescaleDB via standard SQL operations while benefiting from PostgreSQL's mature transaction management, query processing capabilities and indexing mechanisms. The database extends PostgreSQL by introducing optimization mechanisms and data structures specifically designed for time-series workloads. Instead of replacing PostgreSQL functionality, the extension adds specialized support for handling timestamped observations. In the benchmark implementation, environmental observations got stored in a relational table named observations. Each observation was represented as a single database record with spatial, temporal and sensor-related attributes. The repository implementation interacted with TimescaleDB with batch insertion operations and SQL queries. Large ingestion workloads were run using execute_values() function supplied by the PostgreSQL driver.

A basic feature of TimescaleDB is the hypertable. It shows up to users as a normal PostgreSQL table, but internally, it is partitioned into smaller chunks by time interval. This architecture makes storage efficient and supports querying and indexing of large time-series datasets. In the benchmark environment, the observations table was the primary storage structure for environmental sensor data. During initialization, the table was converted into a hypertable using TimescaleDB's hypertable creation functionality. The time attribute of each observation became the partitioning dimension, allowing records to get organized according to their timestamps. This design supports efficiency and optimizes the retrieval of historical data over specific time ranges.

From the perspective of the benchmarking application, data insertion happened via a single logical table. However, internally, TimescaleDB distributed records across many chunks according to their timestamp, enabling scalability. Database initialization scripts were added into the container at startup:

```
volumes:
  - timescale\_data:/var/lib/postgresql/data
  - ./init-timescaledb:/docker-entrypoint-initdb.d
```

The initialization directory allowed database schema creation and TimescaleDB configuration tasks to be executed automatically when the container was first deployed. Persistent storage was provided through the timescale_data Docker volume, to make it possible for benchmark data to be available across container restarts. A health check was configured before execution to verify service availability.

4.3.4 InfluxDB Configuration

InfluxDB was run using its official Docker image as part of the containerized benchmarking environment. The database was automatically configured at startup using environment variables defined in the Docker Compose configuration to make sure that the database could be consistently recreated across different benchmark executions without needing manual setup. InfluxDB differs from traditional relational databases in how it organizes data in buckets and organizations. An organization provides logical grouping for the resources in the database while the bucket works as the primary storage container for time-series records. At initialization, a dedicated bucket and organization were made to store the benchmark datasets used in the study. Authentication was implemented with the token-based security mechanism offered by InfluxDB 2.7. During startup, an access token was auto-generated and supplied to the benchmark application via environment variables. This token was used by the Python repository layer for authentication of all query operations and data ingestion. Token-based authentication made access control simpler while securing communication between the database instance and benchmark framework.

4.3.5 Benchmark Database

Besides the three databases under evaluation, a separate PostgresSQL database was deployed to be the benchmark result repository – BenchmarkDB. It was not included in the performance comparison itself. Its purpose was to create a centralized location to store benchmark execution results generated in the experiments. BenchmarkDB was implemented using PostgresSQL 17 and run as an independent Docker container inside the benchmarking environment. A dedicated data model, IngestionBenchmarkResult, was created to capture performance metrics generated during each execution. The stored information included the dataset identifier, database system under test, batch size, data size, record count, execution time, timestamp of the benchmark execution and throughput measurements.

The benchmark framework inserted results into BenchmarkDB after the completion of each run. This made sure that performance measurements were independently preserved and could be retrieved for subsequent visualization and analysis. The benchmark results were separated to create a consistent and reliable mechanism for collecting experimental data. The stored results were used for visualization, statistical analysis and comparison of database ingestion performance.

4.4 Benchmark Framework Implementation

4.4.1 Configuration Management

A reproducible and flexible configuration management approach was implemented to separate deployment-specific settings from application logic. The benchmark framework used YAML config files alongside environment variables to manage storage locations, database connection parameters, logging settings, and other runtime operations. This allowed the execution of the same codebase across different environments without needing to modify the source code.

Application configuration was stored in YAML files inside the project's configuration directory. Each database system maintained its own configuration file with parameters relevant to the specific technology. The YAML structure separated configuration into logical sections. The general section contains application-level settings like dataset paths and log file locations while the database section has connection parameters needed to establish communication with the database instance. This organization simplifies maintenance as the system grows and improves readability. Using separate config files per database also lowered coupling between benchmark components. It allowed the modification of database-specific settings without affecting other parts of the framework.

Instead of storing sensitive information in the configuration files directly, the framework used environment variables to provide deployment-specific values and credentials. Passwords, ports, database usernames, hostnames, authentication tokens and other parameters were added to the configuration files during startup. This served three purposes; first, sensitive credentials were not added to source code repositories which lowers the risk of accidental disclosure. Second, the same file could be reused across multiple environments like testing, production deployments and development. The only thing needed to change is the environments variable values. Finally, the approach naturally aligned with the Docker-based deployment strategy applied all through the benchmarking framework. Environment variables that were defined in Docker compose could be consumed by the benchmarking application without needing to change.

Configuration loading was done through the load_config() function stored in src/common/config.py. The job of this function was to be the entry point for reading and processing configuration files. The loading process consisted of validating the configuration file path, rendering template variables with Jinja2, resolving environment variable references and parsing the resulting YAML document. Additionally, the configuration framework had a built-in validation logic through the env_var() helper function to retrieve environment variable values and verify that required variables were there before application execution could proceed. The idea of the validation mechanism was to prevent benchmark executions from beginning with incomplete config settings. Missing authentic-

ation tokens, hostnames or credentials were immediately detected during the startup of the application instead of causing failures during benchmark execution. Early validation is especially important in automated benchmarking environments whose experiments run for long and generate large volumes of performance data.

Even though configuration files had a common structure, each database system had its own configuration definition to enable the framework to accommodate different requirements from each database. For example, InfluxDB needed authentication tokens, organization identifiers and bucket names while TimescaleDB needed PostgreSQL connection parameters, user credentials and database names. For reproducibility, all database parameters, authentication settings, storage locations and logging paths were defined explicitly outside the source code and then dynamically loaded at runtime.

4.4.2 Repository Pattern

The benchmarking framework adopted the repository pattern as an abstraction layer between the benchmark execution logic and the database implementations underneath it. This design made it possible for benchmark routines to interact with the databases through consistent operations while isolating database-specific details inside dedicated repository classes. Each repository encapsulated query execution, connection management, data deletion procedures and data insertion as required by each database system. A primary objective of the framework for benchmarking was to compare database performance, not just differences in application implementation. To make this possible, benchmark logic needed to be independent of database-specific APIs and query languages. Without the abstraction layer, the benchmark runner would need different implementations for the different databases, creating code duplicates and making it hard to ensure consistency across all systems.

MongoDB Repository: MongoDB functionality was run through the MongoDBRepo class which extends the MongoDBClient. It provides methods to connect the database, query data, insert documents, manage collections, delete records and perform aggregations. During benchmark execution, collection and database references are retrieved once and stored in memory before timing starts via the connect_and_cache() feature. This helps to reduce Python-side overhead and makes sure that the measured execution time reflects database operations instead of connection management activities. Since MongoDB stores data in the form of JSON-like documents, the repository handles these operations directly via the MongoDB driver while hiding implementation details.

InfluxDB Repository: The functionality for InfluxDB was implemented through InfluxRepo. The class performed an extra transformational step where observation objects are converted into Point objects before insertion. The conversion was implemented through _to_point() which mapped observation attributes to timestamps, fields and tags. Insertion performance gets measured

immediately around the write operation using time.perf_counter_ns().

TimescaleDB Repository: Since TimescaleDB is built on PostgreSQL, observations are stored in a relational table. The repository translates observation objects into SQL-compatible tuples before running insert statements. This implementation supports both batch insertion and single-record operations. Batch insertion is done with PostgreSQL's execute_values() function that allows the insertion of many rows through one SQL statement. Insertion times are measured with high-resolution timers to support precision.

While each repository interacts with different database models, all implementations expose a similar set of capabilities that were needed by the benchmarking framework including batch insertion, database connection, execution time managements and connectivity testing. The repository pattern played a major role in ensuring maintainability and fairness inside the benchmarking framework.

4.4.3 Data Loading Mechanism

The mechanism for locating datasets, loading them into memory and retrieving metadata before execution needed to be consistent. Because the study evaluated database systems using varied datasets, a standardized data loading process was implemented to make sure all databases were benchmarked with equivalent input data. The datasets were stored in database-specific directories arranged by batch size. During execution, the benchmark runner identified the dataset based on its name and selected batch identifier. Dataset naming convention also included the number of observations per dataset. For example, a dataset named batch_8_1000_obs represented the eighth benchmark dataset and contained 1,000 observations. This was done for simplicity and to reduce the risk of loading files incorrectly. Besides loading the dataset, the framework also loaded metadata describing characteristics for each dataset. This was stored in a dataset_sizes.csv file that ensured the dataset size measurements stayed consistent across all benchmark executions.

4.4.4 Benchmark Result Model

All benchmark outcomes get stored in the IngestionBenchmarkResult model which is the central repository for performance measurements from experimental execution. Each benchmark run produces a result record with metadata about the scenario, dataset used, target database system and performance metrics obtained. The model captures the time elapsed, dataset characteristics, throughput metrics, benchmark configuration parameters and operational metrics like the number of performed inserts.

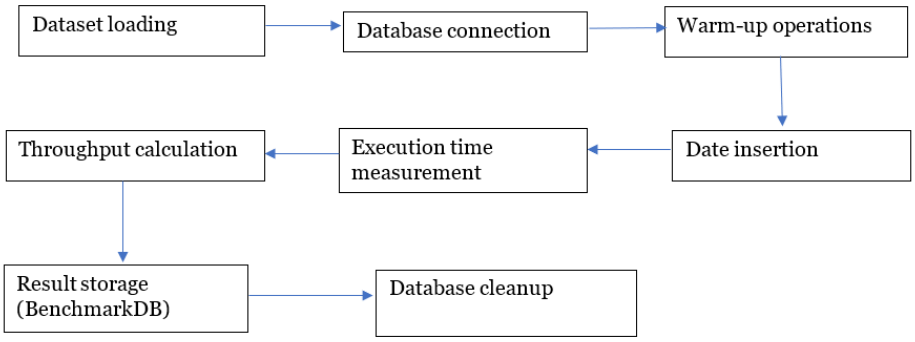

Fig. 4.5: General Ingestion Benchmark Workflow

4.5 Ingestion Benchmark Implementation

The ingestion benchmarking framework was designed to evaluate the write performance of the databases under increasing data volumes. While each database needed a different ingestion implementation because of differences in data models and client APIs, all benchmarks used a common workflow to make sure the results were comparable and consistent. For each run, the appropriate dataset was loaded from the data directory and its metadata was retrieved. The benchmark then connected to the target database and ran a series of warm-up operations before recording any measurements. After the warm-up phase, the benchmark ran database-specific insertion processes while measuring execution time with high-resolution timers. Once insertion was complete, throughput metrics were calculated and the results stored in BenchmarkDBRepo. Finally, the data inserted was removed from the database to make sure the next execution begun from a consistent and clean state.

Figure 4.5 illustrates the overall architecture of the benchmarking framework:

For MongoDB, benchmark datasets were loaded from JSON files. The database stored multiple observations in a single document, each representing a measurement event containing five sensor observations – salinity, conductivity, temperature, dissolved oxygen and turbidity. During execution, datasets were loaded as JSON documents. Since the datasets were defined as observations instead of documents, a conversion step was necessary to determine MongoDB batch sizes. With each document having five observations, a dataset with 1000 observations corresponded to 200 documents. After each benchmark run, all documents were removed from the collection using the cleanup functionality to prevent interference between successive executions.

The TimescaleDB benchmark used a relational representation where each observation was stored as a single row within the observations hypertable. Datasets were parsed and converted into Observation objects just before insertion. The implementation used PostgreSQL's bulk insertion capabilities. Observations were grouped into batches before insertion to improve efficiency and reduce transaction overhead. Insertion time was measured with nanosecond-

resolution timers. After each benchmark run, the observations table was truncated to guarantee an empty database before the next insertion.

The process was similar for InfluxDB. Benchmark datasets were converted into Observation objects with each observation getting transformed into an InfluxDB Point object. Attributes like parameters and sensor identifiers were mapped to fields and tags before getting written to the database. Batch insertion was done using InfluxDB WriteAPI that accepted collection of points and persisted them into a bucket. Similar to the other database systems, execution time measurements were collected with timers positioned immediately around the write operation. After each execution, all measurements were removed.

In all cases, to minimize the impact of startup overheads, a warm-up strategy was used as part of the benchmarking framework. Prior to collecting performance data, each execution ran three preliminary insertion operations which were excluded from the recorded results. The warm-up executions allowed database connections to get fully established, query execution paths to get initialized and internal caches to be populated before timing began. With TimescaleDB, especially, warm-up operations reduced the influence of statement preparation and query planning overheads. For MongoDB and InfluxDB, the warm-ups helped stabilize memory allocation behavior and client-server communication. The results for the initial operations were discarded to make sure the benchmark was measuring steady-state ingestion performance.

4.6 Performance Metrics

To make possible the comparison between InfluxDB, TimescaleDB and MongoDB, the benchmarking framework recorded a set of performance metrics for each benchmark execution. The metrics were directly calculated from the measured execution time and dataset metadata and then stored in BenchmarkDB for analysis. They include:

Execution time – This represents the total time required for the database system to finish a data insertion operation. During benchmarking, the execution time was captured using high-resolution timers. Execution time was the primary measure of ingestion performance. Lower values indicate faster loading capability.

Throughput – Observation throughput is a measure of the number of successfully inserted observations per second. The metric normalizes ingestion performance and allows for comparison across different dataset sizes. The number of observations is the total record count per dataset while the execution time is the measured insertion time in seconds. A higher throughput indicates that the database is able to process a high number of observations per period.

CHAPTER 5

EVALUATION AND RESULTS

This chapter presents the results gotten from the experimental evaluation of TimescaleDB, InfluxDB and MongoDB using the benchmarking framework developed in Chapter 4, as well as their analysis. The design of the experiments was to test the performance characteristics of the three database systems when managing time series data that represents Smart Ocean environmental monitoring workloads. The analysis zooms in on the performance metrics collected during benchmark execution including observation throughput, execution time, storage efficiency, data throughput and query latency. The measurements were obtained by running identical benchmark workloads across database systems within the same deployment environment and dataset conditions. Multiple sizes of datasets were used to evaluate the way performance changed with increases in data volume, allowing an assessment of scalability behavior in addition to raw performance.

This chapter is organized around the primary results generated by the benchmark framework. Ingestion performance gets examined through throughput measurements and execution time. For each database system, the storage requirements are analyzed to evaluate storage efficiency. Subsequently, query benchmark results are presented to assess retrieval performance for representative time series access patterns. The findings are then interpreted in relation to the research objectives of the study and the requirements of large-scale environmental sensor data management. Through this treatment, the chapter seeks to establish the performance trade-offs one has to make with each database technology and to determine how suitable they are to support large-scale time series workloads similar to those generated by the Smart Ocean underwater sensor network.

5.1 Ingestion Performance Results

The objective of the benchmark experiments was to evaluate the ingestion efficiency of each database against an increasing volume of time-series observations like is usually the case with Smart Ocean workload. Benchmark execution fol-

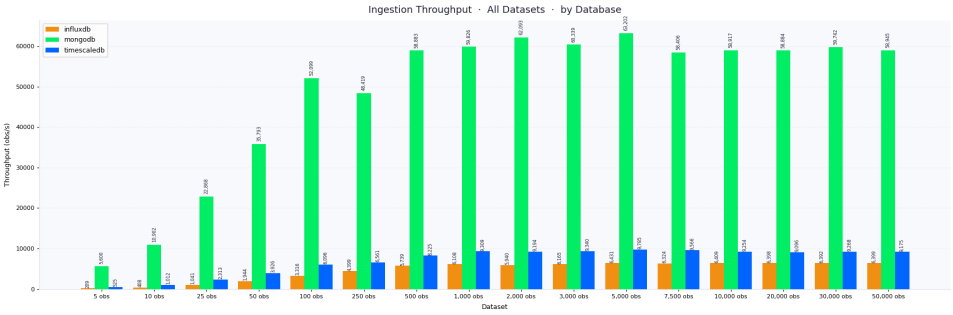

Fig. 5.1: Overview of ingestion throughput results

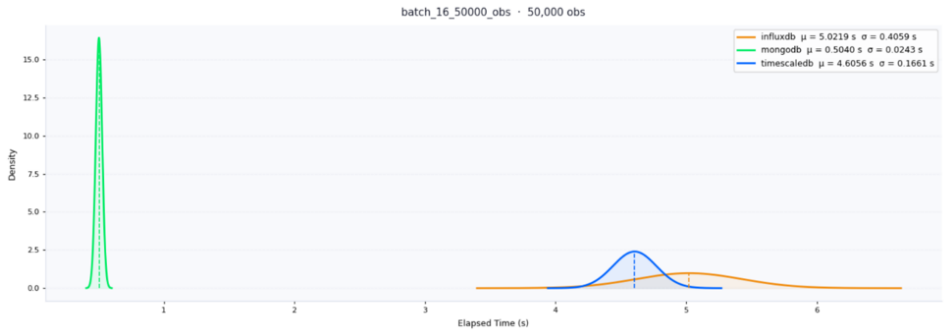

Fig. 5.2: Gaussian distribution plot for Batch 16 (50,000 observations)

lowed the methodology described in the previous chapter where datasets of progressively increasing size got loaded into each database and the insertion performance metrics recorded.

To make the result more reliable, each benchmark configuration was run multiple times and the results were stored in BenchmarkDB for subsequent analysis. The collected results included observation throughput, execution time, dataset size and other metadata necessary to evaluate performance. Statistical analysis was performed on the repeated benchmark runs to find the average performance behavior and variability for each database system. Ingestion results are presented using three performance metrics. Execution time is a measure of the time required to complete dataset insertion operations. Observation throughput is the number of ingested observations in one second and it gives a direct measure of ingestion capacity. Data throughput is an evaluation of the volume of data that gets processed per unit time. It reflects how efficiently each database handles increasing data volumes. These metrics together provide a broad view of scalability and ingestion performance under representative Smart Ocean time-series workloads.

5.1.1 Execution Time

To determine execution time, the experiment measured the elapsed time during the ingestion of a dataset into the target database. Lower execution times show faster ingestion performance and more capability for handling high-volume time-series workloads. Each configuration for the benchmark was run ten times.

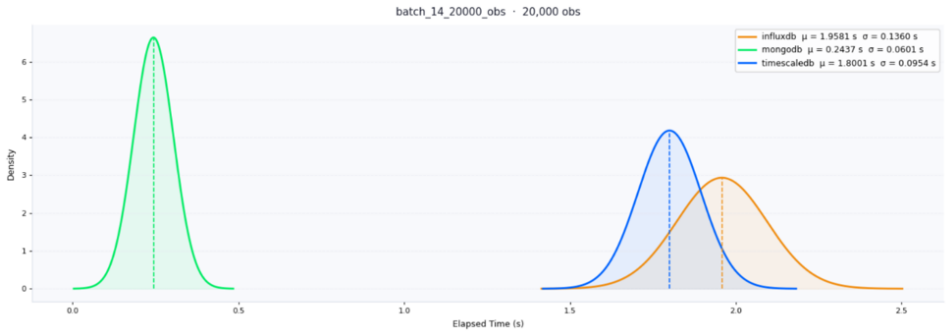

Fig. 5.3: Comparison between the databases at batch 14

The average execution time was calculated to lower the influence of transient system effects and give a more representative performance measure.

Figure 5.1 presents the average execution times gotten across the sixteen benchmark datasets.

The smallest dataset had only five observations and there, MongoDB achieved the lowest average execution time of 0.000895 seconds, compared to 0.009274 seconds for TimescaleDB and 0.021784 seconds for InfluxDB. This trend stayed consistent as dataset sizes increased. MongoDB maintained substantially lower execution times throughout the benchmark series, suggesting that it has very highly efficient write operations for the workload tested.

As the size of the datasets increased to 50 000 observations, execution times went up for all database systems, but the rate of increase was very different for each. MongoDB needed an average of 0.504 seconds to ingest the largest dataset, whereas TimescaleDB required 4.606 seconds and InfluxDB required 5.022 seconds. Based on these figures, MongoDB completed the ingestion workload approximately nine times faster than TimescaleDB and nearly ten times faster than InfluxDB at the largest tested scale.

Variability between repeated benchmark runs was evaluated using the coefficient of variation (CV). MongoDB had higher variability for some intermediate dataset sizes, reaching a maximum CV of 39.28% for the 7,500-observation dataset. In contrast, the largest MongoDB benchmark displayed a CV of only 4.82%, pointing to stable performance at scale. InfluxDB and TimescaleDB in general had lower variability across most dataset sizes even though InfluxDB recorded a coefficient of variation of 29.50% for the 250-observation dataset. Overall, the benchmark results show that all three systems produced reasonably consistent execution times across repeated runs.

Execution-time results suggest that MongoDB provided the most efficient ingestion performance for the evaluated Smart Ocean workload. While TimescaleDB and InfluxDB demonstrated predictable scaling behavior as dataset volume increased, MongoDB consistently achieved lower insertion latencies and maintained this advantage across all benchmark sizes.

5.1.2 Observation Insertion Throughput

Observation throughput measures the number of observations successfully ingested per second and provides a direct indication of the sustained write capacity of each database system. While execution time reflects the total duration required to complete a benchmark run, throughput evaluates how effectively a database converts elapsed time into useful data ingestion work. As dataset sizes increased from 5 observations to 50,000 observations, clear differences emerged in the scalability characteristics of MongoDB, InfluxDB and TimescaleDB.

MongoDB consistently achieved the highest throughput across all dataset sizes. Throughput increased rapidly as dataset sizes grew, rising from 5,638 observations per second for the smallest dataset to approximately 99,428 observations per second for the 50,000-observation dataset. The results indicate that MongoDB benefited substantially from larger write batches, allowing document insertion overhead to be amortized across many observations. Once dataset sizes exceeded approximately 500 observations, MongoDB maintained throughput levels above 80,000 observations per second throughout the remainder of the experiments.

InfluxDB demonstrated a different performance profile. Throughput increased steadily from 230 observations per second for the smallest dataset to approximately 10,000 observations per second for larger datasets. After reaching datasets of around 2,000 observations, throughput stabilized between 9,500 and 10,600 observations per second. This suggests that the write pipeline reached a relatively stable operating region where additional increases in dataset size produced limited throughput gains.

TimescaleDB exhibited behavior similar to InfluxDB at larger scales. Throughput increased progressively from 545 observations per second for the smallest dataset to approximately 10,871 observations per second for the largest dataset. Beyond approximately 1,000 observations, throughput remained relatively stable within the range of 9,900 to 11,100 observations per second. The results indicate that TimescaleDB scaled predictably with increasing workload size and maintained consistent ingestion performance across the larger benchmark datasets.

The largest dataset showed substantial performance differences between MongoDB and the other two database systems. MongoDB delivered nearly an order of magnitude higher ingestion throughput under the benchmark conditions. The throughput results indicate that all three database systems benefited from increased batch sizes, but the magnitude of improvement differed considerably. MongoDB continued scaling aggressively as dataset size increased, whereas InfluxDB and TimescaleDB converged toward stable throughput levels. This behavior suggests that MongoDB's document-oriented write model imposed less overhead during bulk ingestion operations, enabling significantly higher sustained write rates for the Smart Ocean observation

5.1.3 Discussion

Based on the ingestion benchmarks, the database architecture had a significant influence on write performance under the evaluated workload. Even though all three database systems ingested datasets between 5 to 50000 observations successfully, they had markedly different scaling characteristics with the increase in data volume. Notably, performance differences became more pronounced with the growth of the dataset size. For smaller datasets, the absolute execution time differences between the databases were small and would probable not have much practical impact. As the workload increased, MongoDB kept a substantial performance advantage pointing to the fact that its document-oriented storage model and bulk insertion mechanisms were particularly effective for the benchmark dataset structure.

The results also suggest that ingestion performance alone does not necessarily reflect specialization for time-series data. While InfluxDB and TimescaleDB provide capabilities such as time-based organization, retention management, and analytical query support, these architectural features did not translate into superior write performance within the scope of the benchmark. The findings highlight the distinction between optimizing for data ingestion and optimizing for broader time-series management requirements.

Another important outcome is the consistency of system behavior across increasing dataset sizes. None of the evaluated databases exhibited evidence of performance collapse or instability as workload volume increased. Each system converged toward a relatively stable operating region, suggesting that the benchmark sizes remained within the practical capabilities of the deployed environments. This stability increases confidence that the observed performance differences reflect underlying architectural characteristics rather than transient execution anomalies.

Overall, the ingestion experiments indicate that MongoDB delivered the strongest write performance across the evaluated workload, while TimescaleDB and InfluxDB demonstrated similar ingestion characteristics with lower but stable throughput levels. These findings provide a foundation for the subsequent analysis of storage efficiency and read performance, where the relative advantages of the database systems may differ from those observed during data ingestion.

5.2 Query Performance Results

5.2.1 Latest Value Query Latency

Latest-value retrieval is one of the most common operations in environmental monitoring systems because applications frequently require the most recent measurement from a sensor or observation parameter. To evaluate this capability, benchmark queries were executed against each database system for five

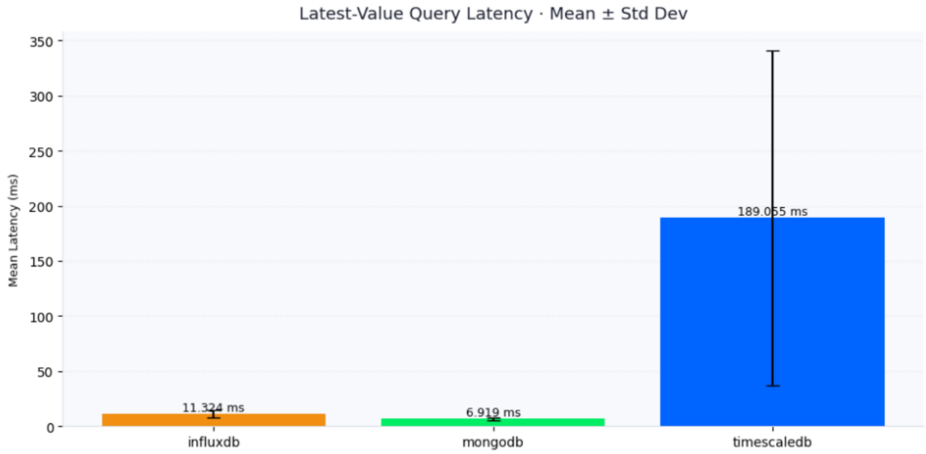

Fig. 5.4: Latest-Value Query Latency – Mean ± Standard Deviation

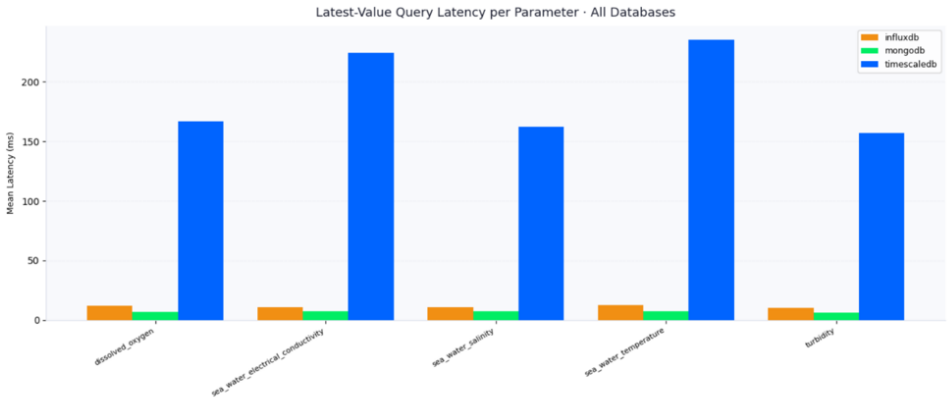

Fig. 5.5: Latest-Value Query Latency per Parameter

Smart Ocean parameters: dissolved oxygen, sea water electrical conductivity, sea water salinity, sea water temperature and turbidity. Each query was executed repeatedly and latency measurements were recorded in milliseconds.

The results revealed substantial differences in query performance between the evaluated database systems. MongoDB achieved the lowest average latency, requiring approximately 6.92 ms to retrieve the latest observation. InfluxDB produced slightly higher latency with an average response time of 11.32 ms. TimescaleDB exhibited considerably higher latency, averaging 189.05 ms across all query executions. Figure 5.4 illustrates the mean latency together with the observed standard deviation for each database system.

The latency differences stayed consistent across all evaluated parameters. As shown in Figure 5.5 , MongoDB consistently returned the fastest responses regardless of the parameter being queried. InfluxDB maintained moderate response times across all parameters, while TimescaleDB recorded substantially higher latencies for every query type. Although the absolute latency varied between parameters, the relative ranking of the database systems remained unchanged throughout the experiments.

The results indicate that MongoDB provided the fastest latest-value retrieval performance under the benchmark workload, followed by InfluxDB. TimescaleDB successfully executed the queries but required substantially more time to return results. For applications requiring frequent retrieval of the most recent sensor measurements, lower query latency may improve responsiveness and reduce delays in downstream monitoring and visualization systems.

5.2.2 *Discussion*

The query benchmark results reveal that database architecture plays a significant role in determining point-query performance for time-series workloads. While all evaluated systems were capable of executing latest-value retrieval operations, the efficiency with which those operations were processed differed considerably. The observed differences suggest that architectural optimizations designed for high-ingestion workloads do not necessarily translate into equivalent performance for low-latency retrieval tasks.

The results also indicate that query latency remained largely independent of the measured sensor parameter. Similar latency patterns were observed across dissolved oxygen, sea water conductivity, sea water salinity, sea water temperature, and turbidity, suggesting that query performance was primarily influenced by database internals rather than characteristics of individual measurements. This consistency strengthens the validity of the benchmark by demonstrating that the observed behaviour was not tied to a particular dataset attribute.

Variability analysis further highlighted differences in operational predictability. Systems exhibiting narrower latency distributions produced more stable response times across repeated executions, while wider distributions indicate greater sensitivity to internal processing overheads and runtime conditions. For analytical applications requiring predictable response behaviour, consistency may be as important as absolute latency performance.

From a Smart Ocean perspective, latest-value queries are commonly used to retrieve the most recent sensor reading for monitoring dashboards, alerting systems, and operational decision support. The benchmark results therefore demonstrate that database selection affects not only data ingestion performance but also the responsiveness of real-time monitoring applications. Consequently, an effective time-series database solution must balance both efficient data ingestion and acceptable query responsiveness rather than optimizing exclusively for a single workload dimension.

The findings from this section complement the ingestion results presented earlier by showing that database performance characteristics vary depending on workload type. A system that performs well under write-intensive conditions may exhibit different behaviour when executing read-oriented operations. For this reason, database evaluation for large-scale time-series environments should consider the complete workload profile rather than relying on a single

performance metric.

5.3 Storage Efficiency Results

This section evaluates the storage efficiency of MongoDB, InfluxDB and TimescaleDB using disk size benchmark results collected during incremental dataset loading. Storage efficiency was assessed by measuring the increase in database size after each dataset insertion and comparing the total storage consumed for an identical volume of Smart Ocean observations. In addition to raw storage growth, the results were normalized using average bytes consumed per observation to provide a fair comparison between database systems.

5.3.1 Storage Size Comparison

Storage consumption was measured by recording the database size before and after each dataset insertion and calculating the resulting storage growth. Across all benchmark datasets, each database stored a cumulative total of 129,440 observations distributed across sixteen progressively larger datasets.

The results revealed substantial differences in storage requirements between the evaluated systems. TimescaleDB exhibited the largest overall storage growth, increasing by approximately 82.58 MB over the course of the benchmark. InfluxDB required considerably less storage, with total growth of approximately 35.44 MB. MongoDB demonstrated the smallest storage footprint, requiring only 0.79 MB of additional storage for the same observation volume.

To account for differences in total storage growth, storage efficiency was further evaluated using average bytes consumed per observation. TimescaleDB required approximately 668.95 bytes per observation, while InfluxDB required approximately 287.12 bytes per observation. MongoDB exhibited the lowest storage cost at approximately 6.38 bytes per observation.

These results indicate that MongoDB achieved substantially higher storage efficiency than the specialized time-series databases under the benchmark conditions employed in this study. While all three systems successfully stored the complete observation workload, the amount of disk space required varied considerably.

5.3.2 Storage Overhead Comparison

The storage benchmark results suggest that database architecture has a significant influence on storage efficiency. Although InfluxDB and TimescaleDB are specifically designed for time-series workloads, both systems consumed substantially more storage than MongoDB during the benchmark. This indicates that additional storage structures maintained by these systems contributed to the observed storage overhead.

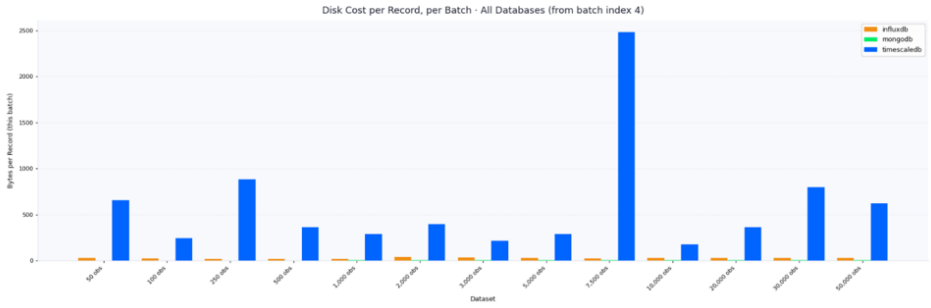

Fig. 5.6: Disk Cost per Record, per Batch – All Databases (from batch index 4)

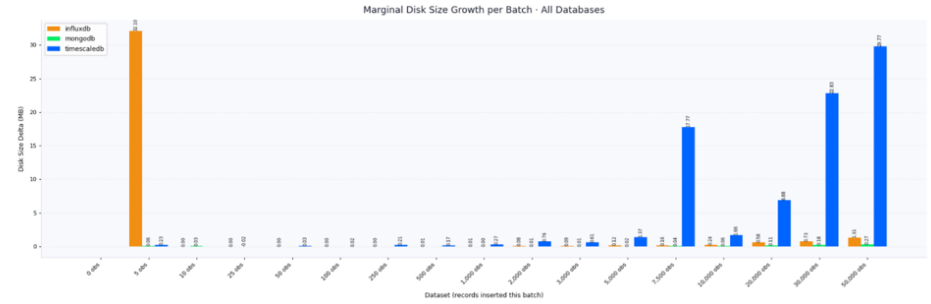

Fig. 5.7: Marginal Disk Size Growth per Batch – All Databases

The grouped disk growth results further show that storage expansion was not perfectly linear across all dataset sizes. Certain insertion batches produced disproportionately larger increases in disk usage, suggesting the allocation of additional internal storage structures during benchmark execution. This behavior was particularly visible in the larger datasets where database files expanded in larger increments rather than at a constant rate.

When storage consumption is normalized on a per-record basis, MongoDB maintained the lowest storage cost across the benchmark workload. InfluxDB consistently occupied less space than TimescaleDB but remained considerably larger than MongoDB. TimescaleDB exhibited the highest storage overhead throughout the experiment, resulting in the largest total storage footprint despite handling the same observation volume.

From a storage efficiency perspective, the benchmark results indicate that MongoDB provided the most compact representation of the Smart Ocean dataset, while InfluxDB achieved intermediate performance and TimescaleDB required the greatest amount of storage capacity. These findings demonstrate that storage optimization strategies differ substantially across database technologies and may influence database selection when long-term retention of large-scale time-series data is required.

5.4 Overall Comparative Analysis

The results obtained from the ingestion, query and storage benchmarks demonstrate that no single database system provided the best performance across

every evaluation criterion. Instead, each technology exhibited strengths that reflected its underlying architectural design and optimization objectives. Consequently, database selection for large-scale time-series applications should be guided by workload requirements rather than by a single performance metric.

5.4.1 MongoDB

MongoDB consistently demonstrated strong overall performance throughout the benchmark evaluation. The database achieved the shortest execution times during ingestion and maintained the highest observation throughput across most dataset sizes. In addition, MongoDB exhibited the lowest storage consumption, resulting in the smallest storage footprint among the evaluated systems.

Query performance results further reinforced MongoDB's strengths. The database returned the lowest latest-value query latencies and maintained relatively stable response times across all tested parameters. The correctness checks also confirmed successful retrieval of the expected results for all benchmark executions.

Taken together, these findings indicate that MongoDB provided the most balanced performance profile across the evaluated workload. Its ability to combine high ingestion performance, low query latency and efficient storage utilization suggests that document-oriented architectures can be highly competitive for time-series data management despite not being exclusively designed as time-series databases.

5.4.2 InfluxDB

InfluxDB demonstrated performance characteristics that positioned it between MongoDB and TimescaleDB in most benchmark categories. While its ingestion performance was lower than MongoDB, it remained competitive across increasing dataset sizes and maintained relatively stable throughput as workload volume increased.

The query benchmarks showed that InfluxDB consistently achieved lower latency than TimescaleDB, although it did not match MongoDB's response times. Latency variability was also moderate compared to the significantly larger fluctuations observed in TimescaleDB. From a storage perspective, InfluxDB required substantially less disk space than TimescaleDB while consuming more storage than MongoDB. This intermediate storage behaviour suggests a trade-off between specialized time-series functionality and storage efficiency.

Overall, InfluxDB exhibited a balanced performance profile with no extreme weaknesses. Its results indicate that purpose-built time-series databases can provide competitive ingestion and query performance while maintaining reasonable storage requirements for large-scale sensor workloads.

5.4.3 TimescaleDB

TimescaleDB exhibited the most distinctive performance profile among the evaluated systems. Although it successfully handled all benchmark workloads and returned correct query results, its behavior differed substantially from both MongoDB and InfluxDB.

The query benchmarks revealed significantly higher latest-value query latencies and greater performance variability. The latency distributions indicated that response times were less predictable than those observed in the other database systems. Similarly, ingestion performance generally lagged behind MongoDB and remained comparable to or below InfluxDB for many dataset sizes.

Storage measurements also showed that TimescaleDB generated the largest overall storage footprint and the highest storage cost per observation. While this behavior may be associated with the relational structures and storage mechanisms employed by the database, the benchmark results demonstrate that these characteristics resulted in greater disk utilization under the evaluated workload.

Despite these observations, TimescaleDB maintained consistent operation throughout the experiments and successfully supported both data ingestion and retrieval tasks. Its results illustrate that relational time-series extensions may prioritize capabilities beyond the performance metrics evaluated in this study, resulting in different trade-offs when compared with document-oriented and purpose-built time-series database architectures.

When the benchmark results are considered collectively, MongoDB emerged as the strongest overall performer for the Smart Ocean workload used in this research. It combined superior ingestion performance, low query latency and efficient storage utilization while maintaining correct query results throughout the experiments.

InfluxDB generally occupied a middle position across the evaluated metrics, offering competitive performance without exhibiting the pronounced storage or latency costs observed in TimescaleDB. TimescaleDB, while functionally successful, demonstrated less favourable results for the specific workload characteristics represented by the benchmark datasets.

The findings suggest that workload requirements play a critical role in database selection. For environments where high ingestion rates, low-latency retrieval and storage efficiency are primary concerns, MongoDB provided the most favourable performance characteristics under the conditions evaluated in this study. InfluxDB represented a viable alternative with balanced behavior across multiple metrics, while TimescaleDB exhibited trade-offs that may be acceptable in scenarios where capabilities not assessed in this benchmark are of greater importance.

5.4.4 Answering the Research Questions

This study sought to evaluate the suitability of selected database technologies for managing large-scale time-series data generated by the Smart Ocean underwater sensor network. The findings obtained from the benchmark experiments provide answers to the research questions as follows.

Research Question 1: What does the Smart Ocean time-series workload require in terms of ingestion rate, data volume, scalability, consistency, query patterns and retention duration?

The Smart Ocean workload is characterized by the continuous generation of sensor observations that must be stored efficiently and made available for subsequent analysis. Such workloads require database systems capable of supporting sustained data ingestion, efficient storage utilization, and rapid retrieval of recently collected measurements. The benchmark datasets used in this study represented progressively increasing observation volumes, allowing the behavior of each database to be evaluated under growing data loads. The results demonstrate that ingestion performance, storage efficiency and latest-value query responsiveness are critical requirements for supporting this type of workload.

Research Question 2: How do the architectural designs of TimescaleDB, InfluxDB and MongoDB influence their expected performance characteristics for time-series workloads?

The benchmark results indicate that database architecture has a direct influence on observed performance. MongoDB's document-oriented architecture enabled high ingestion throughput, low query latency and efficient storage utilization across the evaluated datasets. InfluxDB's specialized time-series architecture provided competitive ingestion and query performance while maintaining moderate storage requirements. TimescaleDB, which extends a relational database architecture with time-series capabilities, successfully supported the workload but exhibited higher query latency and greater storage consumption. These findings demonstrate that architectural design decisions influence the trade-offs between ingestion performance, retrieval speed and storage efficiency.

Research Question 3: How do the selected database systems compare with respect to ingestion throughput, query latency and storage efficiency under representative Smart Ocean workload conditions?

The benchmark results show clear performance differences among the evaluated database systems. MongoDB achieved the highest ingestion throughput, the lowest latest-value query latency and the smallest storage footprint. InfluxDB generally ranked second across the evaluated metrics, providing balanced performance without extreme weaknesses. TimescaleDB demonstrated the lowest overall performance in terms of ingestion speed, query latency and storage ef-

ficiency for the tested workload. Based on the benchmark results, MongoDB provided the most favourable combination of performance characteristics for the Smart Ocean time-series datasets used in this study.

The benchmark results presented in this chapter demonstrate clear differences in ingestion performance, query responsiveness and storage efficiency across the evaluated database systems. These findings provide the empirical basis for the conclusions and recommendations presented in Chapter 6, where the implications of the results are discussed in relation to Smart Ocean deployment requirements and future research directions.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

This chapter presents the conclusions drawn from the study and discusses their implications for the management of large-scale time-series data generated by the Smart Ocean underwater sensor network. The study designed and implemented a benchmarking framework to evaluate the ingestion performance, query performance and storage efficiency of MongoDB, InfluxDB and TimescaleDB under representative environmental observation workloads. Based on the experimental results presented in Chapter 5, the chapter summarizes the major findings in relation to the research objectives and research questions of the study. It also discusses the limitations of the benchmarking approach and identifies areas where further investigation may be required. Finally, recommendations are provided for database selection in Smart Ocean environments and for future research aimed at extending the evaluation of time-series database technologies for environmental monitoring applications.

6.1 Conclusions

The goal of this study was to evaluate the suitability of selected database technologies for managing the large volumes of time-series data generated by the Smart Ocean underwater sensor network. As environmental monitoring systems continue to produce increasing quantities of timestamped sensor observations, selecting an appropriate database platform becomes an important consideration for ensuring efficient data storage, retrieval and long-term management. To address this challenge, the study designed and implemented a benchmarking framework capable of evaluating the performance of MongoDB, InfluxDB and TimescaleDB under representative environmental observation workloads.

A major outcome of the research was the successful development and implementation of a reproducible benchmarking framework that enabled consistent evaluation across the three database systems. The framework provided a controlled environment in which identical datasets could be ingested, queried and analyzed under equivalent experimental conditions. Through the use of standardized datasets, common performance metrics, containerized deployment and

automated benchmark execution, the framework generated a reliable basis for comparing the behavior of the evaluated database technologies. The implementation demonstrated that it is possible to perform systematic benchmarking of heterogeneous database systems while maintaining consistency in workload execution and result collection.

The benchmark results revealed that database architecture had a significant influence on performance outcomes. Although all three database systems successfully stored and retrieved the benchmark datasets, substantial differences were observed in ingestion performance, query latency and storage efficiency. These differences became more pronounced as dataset sizes increased, indicating that the internal design of a database plays an important role in determining how effectively it scales under growing time-series workloads.

Among the evaluated systems, MongoDB consistently demonstrated the strongest performance across the metrics examined in this study. The database achieved the shortest execution times during data ingestion, the highest observation throughput and data throughput values, the lowest latest-value query latencies and the smallest storage footprint. These results indicate that MongoDB was highly effective at handling the environmental observation workloads used in the benchmark. Its document-oriented architecture enabled efficient storage and retrieval of the benchmark data and allowed it to maintain strong performance as dataset sizes increased. Within the scope of the evaluated workloads, MongoDB provided the most favourable overall combination of ingestion efficiency, query responsiveness and storage utilization.

InfluxDB demonstrated competitive and balanced performance throughout the experiments. While it did not achieve the ingestion throughput or storage efficiency observed in MongoDB, it maintained stable behavior across increasing dataset sizes and consistently produced lower query latencies than TimescaleDB. The benchmark results showed that InfluxDB scaled predictably as workload volume increased and maintained relatively consistent performance across repeated benchmark executions. These findings suggest that InfluxDB provides a balanced approach to managing time-series data and remains a viable option for environmental monitoring applications requiring dependable ingestion and retrieval performance.

TimescaleDB successfully supported all benchmark workloads and correctly executed both ingestion and query operations. However, under the conditions evaluated in this study, it generally exhibited longer execution times, higher latest-value query latencies and greater storage requirements than the other database systems. Although TimescaleDB remained operationally stable throughout the experiments, its performance profile reflected different trade-offs compared with MongoDB and InfluxDB. The results indicate that extending a relational database architecture with time-series capabilities may involve additional overheads that affect ingestion performance, query responsiveness and storage utilization for the workload considered in this research.

The study also demonstrated that performance evaluation of time-series databases should not rely on a single metric. Databases that perform well in one area may exhibit different behavior when evaluated against other workload requirements. The results showed that ingestion performance, query latency and storage efficiency each provide a different perspective on database suitability. Consequently, database selection decisions should be guided by the specific operational requirements of the target application rather than by a single measure of performance.

Overall, the findings indicate that all three database systems were capable of supporting Smart Ocean environmental observation data, but they did so with different performance characteristics and trade-offs. Within the scope of the benchmark implementation and the performance metrics evaluated, MongoDB emerged as the strongest overall performer, InfluxDB demonstrated balanced and competitive behavior, and TimescaleDB provided functional support for the workload while exhibiting higher storage and latency costs. The study therefore concludes that database architecture significantly influences the efficiency with which large-scale time-series workloads are managed and that careful evaluation of workload requirements is essential when selecting a database platform for environmental monitoring systems such as Smart Ocean.

6.2 Contributions of the Study

This study contributes to the growing body of knowledge on the management of large-scale time-series data by providing both a practical benchmarking framework and empirical evidence regarding the performance characteristics of selected database technologies under environmental monitoring workloads. The contributions of the research span methodological, technical and practical dimensions

The first contribution is the design and implementation of a reproducible benchmarking framework for evaluating time-series database performance. The framework provided a standardized environment for executing ingestion benchmarks, collecting performance measurements and storing benchmark results. Through the use of a common execution workflow, database abstraction mechanisms and containerized deployment, the framework enabled MongoDB, InfluxDB and TimescaleDB to be evaluated under equivalent conditions. This implementation provides a foundation that can be extended for future benchmarking studies involving additional databases, larger datasets or alternative workload patterns.

The second contribution is the development of a representative benchmark workload based on Smart Ocean environmental observation data. Synthetic datasets were generated using a common observation model that captured the temporal, spatial and sensor characteristics of marine monitoring systems. By creating datasets of progressively increasing size and transforming them into

formats appropriate for each database system, the study established a controlled mechanism for assessing how database performance changes with increasing data volume. The resulting workload provides a practical approximation of the challenges associated with storing and managing environmental sensor observations.

The third contribution is the generation of empirical performance data comparing MongoDB, InfluxDB and TimescaleDB under identical experimental conditions. The study produced quantitative evidence relating to ingestion performance, execution time, observation throughput, data throughput, storage efficiency and latest-value query latency. By evaluating all three database systems using the same datasets, deployment environment and benchmark methodology, the research provides a consistent basis for understanding the performance trade-offs associated with different database architectures.

The fourth contribution is the provision of evidence that can support database selection for large-scale time-series applications. The benchmark results demonstrate how document-oriented, relational time-series and purpose-built time-series database architectures behave when managing environmental observation data. The findings offer practical insights for researchers, engineers and system designers who must select database technologies for applications involving continuous sensor data generation, environmental monitoring and similar time-series workloads.

Finally, the study contributes to the Smart Ocean research initiative by providing an initial performance evaluation of database technologies capable of supporting underwater sensor network data management. The findings establish a baseline for future investigations into scalable storage solutions for environmental monitoring systems and provide a foundation for further research into the management of large-scale time-series data within Smart Ocean and related domains.

6.3 Limitations of the Study

While the study achieved its objectives and generated useful insights into the performance characteristics of MongoDB, InfluxDB and TimescaleDB, several limitations should be acknowledged when interpreting the results.

First, the evaluation was limited to three database technologies. Although MongoDB, InfluxDB and TimescaleDB represent widely used approaches to time-series data management, there are other database systems that were not included in the study. Consequently, the findings should be interpreted as a comparison among the selected technologies rather than as a comprehensive evaluation of all available time-series database solutions.

Second, the benchmark focused on a limited set of workload types. The evaluation concentrated primarily on data ingestion performance, storage efficiency

and latest-value query latency. Other workload characteristics that may be important in operational environments, such as complex analytical queries, time-range aggregations, concurrent access patterns, data compression, retention management and long-term archival performance, were not included within the scope of the benchmark. As a result, the findings reflect performance under the specific workloads evaluated rather than all possible usage scenarios.

Third, the experiments were conducted within a single deployment environment using a fixed hardware and software configuration. Although containerization helped ensure consistency and reproducibility across benchmark executions, performance characteristics may vary when databases are deployed on different hardware platforms, operating systems or distributed infrastructures. The results therefore represent the behavior of the evaluated systems under the experimental conditions used in this study.

Fourth, the benchmark datasets were generated from synthetic environmental observations designed to represent Smart Ocean workloads. While this approach ensured consistency, repeatability and controlled dataset sizes, synthetic data may not fully capture all characteristics of real-world sensor streams, including irregular sampling intervals, missing values, data anomalies and varying observation patterns.

Finally, resource utilization metrics such as CPU usage, memory consumption, disk I/O activity and network utilization were not included in the final benchmark scope. The study therefore evaluated database performance primarily through execution time, throughput, query latency and storage efficiency. Additional insight into the operational costs of each database system could be obtained by incorporating resource utilization measurements into future evaluations.

Despite these limitations, the study provides a consistent and reproducible comparison of the selected database systems and offers valuable evidence regarding their performance characteristics under representative environmental monitoring workloads.

6.4 Recommendations for Future Work

The findings of this study provide a foundation for further research into the evaluation and selection of database technologies for large-scale time-series data management. While the benchmarking framework successfully compared MongoDB, InfluxDB and TimescaleDB using representative Smart Ocean workloads, several opportunities exist to extend and enhance the scope of future investigations.

One area for future work is the evaluation of additional database technologies. The current study focused on three widely used database systems representing document-oriented, purpose-built time-series and relational time-series archi-

tectures. Future studies could expand the comparison to include other platforms such as QuestDB, ClickHouse, Apache Druid, VictoriaMetrics or other emerging time-series data management solutions. Including a broader range of technologies would provide a more comprehensive understanding of the database landscape and offer additional guidance for practitioners selecting storage solutions for environmental monitoring systems.

Future research should also incorporate a wider variety of workload types. The benchmark developed in this study focused primarily on ingestion performance, storage efficiency and latest-value query latency. While these are important aspects of Smart Ocean data management, operational systems frequently perform more complex analytical tasks. Additional benchmark workloads could include time-range queries, aggregation operations, moving averages, statistical summaries, trend analysis and multi-parameter retrieval scenarios. Evaluating these workloads would provide a more complete picture of database performance across the full lifecycle of time-series data processing.

Another important direction is the investigation of distributed and clustered deployments. The benchmark experiments were conducted using single-node database instances to ensure consistency and simplify performance comparisons. However, large-scale environmental monitoring systems may eventually require distributed architectures to support higher ingestion rates, larger storage capacities and increased availability. Future studies could evaluate how database performance changes when deployed across multiple nodes and assess scalability, fault tolerance and load distribution characteristics under distributed operating conditions.

Resource utilization should also be incorporated into future benchmarking efforts. The current study evaluated performance using execution time, throughput, query latency and storage efficiency. While these metrics provide valuable insight into database behavior, they do not capture the computational resources required to achieve that performance. Future work should include measurements of CPU utilization, memory consumption, disk I/O activity and network usage. Such measurements would enable a more complete assessment of operational efficiency and help identify trade-offs between performance and infrastructure requirements.

Future studies could further investigate database behavior under concurrent workloads. Real-world environmental monitoring systems often support multiple users, applications and services accessing data simultaneously while new observations continue to be ingested. Benchmarking under concurrent read and write workloads would provide insight into how database performance changes under realistic operational conditions and would help evaluate system responsiveness in multi-user environments.

The benchmark datasets used in this study were generated synthetically to ensure repeatability and control over workload size. Future research could

complement these experiments with real-world Smart Ocean datasets collected from operational sensor deployments. Evaluating database performance using real environmental observations would help determine whether the performance characteristics observed in this study remain consistent under more complex and potentially irregular data generation patterns.

Finally, the benchmarking framework developed in this research could be extended into a more comprehensive evaluation platform. Additional benchmark modules could be added to assess data compression effectiveness, retention policy management, backup and recovery performance, long-term archival storage behavior and advanced analytical capabilities. Such extensions would broaden the applicability of the framework and support more comprehensive evaluations of database technologies for environmental monitoring and other large-scale time-series applications.

By pursuing these research directions, future studies can build upon the foundation established in this thesis and develop a deeper understanding of how database architectures perform under the diverse requirements of modern time-series data management systems. Collectively, these efforts would contribute to more informed database selection decisions and support the continued development of scalable data management solutions for Smart Ocean and similar environmental monitoring initiatives.

BIBLIOGRAPHY

[1] Sfi smart ocean - flexible and cost-effective monitoring for management of a healthy and productive ocean - prosjektbanken. URL https://prosjektbanken.forskningsradet.no/project/FORISS/309612. Accessed: 2026-02-27. 1

[2] About – sfi smart ocean. URL https://sfismartocean.no/about/. Accessed: 2026-02-28. 1.1.1

[3] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. Column-stores vs. row-stores: How different are they really? In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, SIGMOD '08, pages 967–980, Vancouver, Canada, 2008. ACM. doi: 10.1145/1376616.1376712. URL https://www.cs.umd.edu/~abadi/papers/abadi-sigmod08.pdf. 2.2.3

[4] Sadiq H. Abdulhussain, Basheera M. Mahmmod, Almuntadher Alwhelat, Dina Shehada, Zainab I. Shihab, Hala J. Mohammed, Tuqa H. Abdulameer, Muntadher Alsabah, Maryam H. Fadel, Susan K. Ali, Ghadeer H. Abbood, Zianab A. Asker, and Abir Hussain. A comprehensive review of sensor technologies in iot: Technical aspects, challenges, and future directions. *Computers*, 14(8), 2025. ISSN 2073-431X. doi: 10.3390/computers14080342. URL https://www.mdpi.com/2073-431X/14/8/342. 2

[5] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. Blinkdb: Queries with bounded errors and bounded response times on very large data. In *Proceedings of the 8th ACM European Conference on Computer Systems*, EuroSys '13, pages 29–42. ACM, 2013. doi: 10.1145/2465351.2465355. URL https://arxiv.org/pdf/1203.5485. 2.4.5

[6] Shivnath Babu and Jennifer Widom. Continuous queries over data streams. *ACM SIGMOD Record*, 30(3):109–120, 2001. doi: 10.1145/603867.603884. URL http://web.cs.wpi.edu/~mukherab/i/CQ.pdf. 2.4.5

[7] Siddhartha Bhandari, Neil Bergmann, Raja Jurdak, and Branislav Kusy. Time series data analysis of wireless sensor network measurements of temperature. *Sensors 2017, Vol. 17, Page 1221*, 17:1221, 5 2017. ISSN 14248220. doi: 10.3390/s17061221. URL https://www.mdpi.com/1424-8220/17/6/1221/htmhttps://www.mdpi.com/1424-8220/17/6/1221.

[8] Alan Brown and Kurt Wallnau. Framework for evaluating software technology. *Software, IEEE*, 13:39 – 49, 10 1996. doi: 10.1109/52.536457. URL https://ieeexplore.ieee.org/document/536457. 1.4, 1.1, 3.1, 6.4

[9] ClickHouse. Overview of clickhouse architecture. ClickHouse Documentation, 2026. URL https://clickhouse.com/docs/development/architecture. Accessed 2026-08-20. 2.2.3, 2.3

[10] Juan A. Colmenares, Reza Dorrigiv, and Daniel G. Waddington. Ingestion, indexing and retrieval of high-velocity multidimensional sensor data on a single node, 2017. URL https://arxiv.org/abs/1707.00825. 2

[11] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, page 143–154, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450300360. doi: 10.1145/1807128.1807152. URL https://doi.org/10.1145/1807128.1807152. 2, 2.5.2

[12] Feliciano Pedro Francisco Domingos, Ahmad Lotfi, Isibor Kennedy Ihianle, Omprakash Kaiwartya, and Pedro Machado. Underwater communication systems and their impact on aquatic life—a survey. *Electronics*, 14(1), 2025. ISSN 2079-9292. doi: 10.3390/electronics14010007. URL https://www.mdpi.com/2079-9292/14/1/7. 1

[13] Emad Felemban, Faisal Karim Shaikh, Umair Mujtaba Qureshi, Adil A. Sheikh, and Saad Bin Qaisar. Underwater sensor network applications: A comprehensive survey. *International Journal of Distributed Sensor Networks*, 11(11):896832, 2015. doi: 10.1155/2015/896832. URL https://doi.org/10.1155/2015/896832. 1

[14] Thanaa M. Ghanem, Moustafa A. Hammad, Mohamed F. Mokbel, Walid G. Aref, and Ahmed K. Elmagarmid. Incremental evaluation of sliding-window queries over data streams. *IEEE Transactions on Knowledge and Data Engineering*, 19(1):57–72, 2007. doi: 10.1109/TKDE.2007.250583. URL https://www.cs.purdue.edu/homes/ake/pub/TKDE-0148-0405-1.pdf. 2.4.5

[15] Graphite Project. Overview. Graphite Documentation. URL https://graphite.readthedocs.io/en/latest/overview.html. Accessed: 16 March 2026. 2.2.5

[16] Piotr Grzesik and Dariusz Mrozek. Comparative analysis of time series databases in the context of edge computing for low power sensor networks. *Computational Science – ICCS 2020*, 12141:371, 2020. ISSN 16113349. doi: 10.1007/978-3-030-50426-7_28. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC7302557/. 2, 2.1.1, 2.1.4

[17] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. ISSN 0167-739X. doi: https://doi.org/10.1016/j.future.2013.01.010. URL https://www.sciencedirect.com/science/article/pii/

S0167739X13000241. Including Special sections: Cyber-enabled Distributed Computing for Ubiquitous Cloud and Network Services and Cloud Computing and Scientific Applications — Big Data, Scalable Analytics, and Beyond. 2.1.2, 2.1.3

[18] IIoT World. Long-term data retention in iiot with time series databases. IIoT World, 2024. URL https://www.iiot-world.com/predictive-analytics/predictive-maintenance/long-term-iiot-data-retention-with-time-series-databases/. Accessed 2026-08-20. 2.1.2, 2.1.3, 2.3

[19] InfluxData. In-memory indexing and the time-structured merge tree (tsm). Technical report, InfluxData Documentation, 2019. URL https://docs.influxdata.com/influxdb/. 2.2.2

[20] InfluxData. influxdb: Scalable datastore for metrics, events, and real-time analytics. GitHub, 2026. URL https://github.com/influxdata/influxdb. Accessed 2026-08-20. 3.2

[21] InfluxData. Influxdb key concepts. InfluxDB OSS v1 Documentation, 2026. URL https://docs.influxdata.com/influxdb/v1/concepts/key_concepts/. Accessed 2026-08-20. 2.4.1

[22] InfluxData, Inc. In-memory indexing and the time-structured merge tree (tsm). InfluxDB OSS v1 Documentation. URL https://docs.influxdata.com/influxdb/v1/concepts/storage_engine/. Accessed: 16 March 2026.

[23] Søren Kejser Jensen, Torben Bach Pedersen, and Christian Thomsen. Time series management systems: A survey. *CoRR*, abs/1710.01077, 2017. URL http://arxiv.org/abs/1710.01077. 1, 1.1.2, 2.1.2, 2.1.3, 2.3, 2.4.2

[24] Abdelouahab Khelifati, Mourad Khayati, Philippe Cudré-Mauroux, Djellel Hached, and Pierre Bourhis. Tsm-bench: Benchmarking time series database systems for monitoring applications. *Proceedings of the VLDB Endowment*, 16(11):3363–3376, 2023. doi: 10.14778/3611479.3611533. URL https://www.vldb.org/pvldb/vol16/p3363-khelifati.pdf. 2.5.1, 2.5.3, 3.6.6

[25] Muhammad Adeel Mahmood, Winston K.G. Seah, and Ian Welch. Reliability in wireless sensor networks: A survey and challenges ahead. *Computer Networks*, 79:166–187, 2015. ISSN 1389-1286. doi: https://doi.org/10.1016/j.comnet.2014.12.016. URL https://www.sciencedirect.com/science/article/pii/S1389128614004708. 2.1.3

[26] Janot Mendler de Suarez, Biliana Cicin-Sain, Kateryna Wowk, Rolph Payet, and Ove Hoegh-Guldberg. Ensuring survival: Oceans, climate and security. *Ocean and Coastal Management*, 90:27–37, 2014. ISSN 0964-5691. doi: https://doi.org/10.1016/j.ocecoaman.2013.08.007. URL https://www.sciencedirect.com/science/article/pii/S0964569113001877. 1

[27] MongoDB. Time series — database manual. MongoDB Documentation, 2026. URL https://www.mongodb.com/docs/manual/core/timeseries-collections/. Accessed 2026-08-20. 3.1, 6.4

[28] MongoDB. mongo: The mongodb database. GitHub, 2026. URL https://github.com/mongodb/mongo. Accessed 2026-08-20. 3.2

[29] MongoDB Documentation Team. Time series collections. https://www.mongodb.com/docs/manual/core/timeseries-collections/, n.d. Accessed: 2026-03-06. 2.2.4

[30] MongoDB, Inc. Documents. MongoDB Manual. URL https://www.mongodb.com/docs/manual/core/document/. Accessed: 16 March 2026. 2.2.4

[31] Mohammad Nasar and Mohammad Abu Kausar. Suitability of influxdb database for iot applications. *International Journal of Innovative Technology and Exploring Engineering*, 8(10):1850–1857, August 2019. doi: 10.35940/ijitee.j9225.0881019. URL http://dx.doi.org/10.35940/ijitee.J9225.0881019. 2.2.2

[32] Vazha Omanashvili. Json generator — tool for generating random data. json-generator.com, 2026. URL https://json-generator.com/. Accessed 2026-08-20. 4.2.2

[33] Yosafe Fesaha Oqbamecail. timeseries-thesis: Research data. GitHub, 2026. URL https://github.com/Yo581515/timeseries-thesis/tree/main/data. Accessed 2026-08-20. B

[34] Yosafe Fesaha Oqbamecail. timeseries-thesis. GitHub, 2026. URL https://github.com/Yo581515/timeseries-thesis. Accessed 2026-08-20. A

[35] Carlos Ordonez and Ladjel Bellatreche. A survey on parallel database systems from a storage perspective: Rows versus columns. In Mourad Elloumi et al., editors, *Database and Expert Systems Applications*, volume 903 of *Communications in Computer and Information Science*, pages 5–20, Cham, 2018. Springer. doi: 10.1007/978-3-319-99133-7_1. URL https://www2.cs.uh.edu/~ordonez/pdfwww/w-2018-DEXA-paralleldbms.pdf. 2.2.1

[36] Tuomas Pelkonen, Scott Franklin, Paul Cavallaro, Qi Huang, Justin Meza, Justin Teller, and Kaushik Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *Proceedings of the VLDB Endowment*, 8(12): 1816–1827, 2015. doi: 10.14778/2824032.2824078. URL https://www.vldb.org/pvldb/vol8/p1816-teller.pdf. 2.2.2, 2.1, 6.4

[37] PostgreSQL Global Development Group. Table partitioning. PostgreSQL Documentation, 2025. URL https://www.postgresql.org/docs/current/ddl-partitioning.html. Accessed 2026-08-20. 2.1.3, 2.2.1

[38] Prometheus Authors. Storage. Prometheus Documentation. URL https:

//prometheus.io/docs/prometheus/latest/storage/. Accessed: 16 March 2026. 2.2.5

[39] Prometheus Overview. Prometheus: Overview. https://prometheus.io/docs/introduction/overview/, n.d. Accessed: 2026-03-06. 2.2.5

[40] QuestDB. The best time series databases compared. QuestDB Blog, 2026. URL https://questdb.com/blog/best-time-series-databases/. Accessed 2026-08-20. 3.1, 6.4

[41] Mohit Srivastava. Architectural evolution and selection framework for database systems in ai-ready data platforms. *arXiv preprint arXiv:2606.08317*, 2026. URL https://arxiv.org/pdf/2606.08317. 2.1.2, 2.3

[42] Michael Stonebraker, Uundefinedur Çetintemel, and Stan Zdonik. The 8 requirements of real-time stream processing. *SIGMOD Rec.*, 34(4):42–47, December 2005. ISSN 0163-5808. doi: 10.1145/1107499.1107504. URL https://doi.org/10.1145/1107499.1107504. 2.1.1

[43] Paula Ta-Shma, Guy Khazma, Gal Lushi, and Oshrit Feder. Extensible data skipping, 2020. URL https://arxiv.org/abs/2009.08150. 2.4.2

[44] Toni Taipalus. Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, 208:111872, February 2024. ISSN 0164-1212. doi: 10.1016/j.jss.2023.111872. URL http://dx.doi.org/10.1016/j.jss.2023.111872. 2.2.4, 2.5.1

[45] Tiger Data (Timescale). The best time series databases compared. Tiger Data Learn, 2026. URL https://www.tigerdata.com/learn/the-best-time-series-databases-compared. Accessed 2026-08-20. 3.1, 6.4

[46] Tiger Data (Timescale). Understand continuous aggregates. TimescaleDB Documentation, 2026. URL https://www.tigerdata.com/docs/learn/continuous-aggregates. Accessed 2026-08-20. 2.4.5

[47] Tiger Data (Timescale). Designing your database schema: Narrow, medium or wide table layout. Tiger Data Learn, 2026. URL https://www.tigerdata.com/learn/designing-your-database-schema-wide-vs-narrow-postgres-tables. Accessed 2026-08-20. 2.4.1

[48] Timescale. timescaledb: A time-series database for high-performance real-time analytics packaged as a postgres extension. GitHub, 2026. URL https://github.com/timescale/timescaledb. Accessed 2026-08-20. 3.2

[49] Timescale. tsbs: Time series benchmark suite, a tool for comparing and evaluating databases for time series data. GitHub, 2026. URL https://github.com/timescale/tsbs. Accessed 2026-08-20.

[50] Timescale, Inc. Timescaledb, 2017. URL https://github.com/timescale/timescaledb. Accessed: 16 March 2026. 2.2.1

[51] Andrea Zanella, Nicola Bui, Angelo Castellani, Lorenzo Vangelista, and Michele Zorzi. Internet of things for smart cities. *IEEE Internet of Things Journal*, 1(1):22–32, 2014. doi: 10.1109/JIOT.2014.2306328. URL https://ieeexplore.ieee.org/document/6740844.

APPENDIX A

SOURCE CODE

The source code developed for this thesis is available in the project repository [34].

APPENDIX B

RESEARCH DATA

The research data used in this thesis is available in the project repository [33].

List of Figures

List of Tables